

ASSIGNMENT FINAL REPORT

Qualification	Pearson BTEC Level 5 Higher National Diploma in Computing		
Unit number and title	Unit 7: Software Development Life Cycle		
Submission date		Date Received 1st submission	
Re-submission Date		Date Received 2nd submission	
Student Name	SIN VAN HUYNH	Student ID	SVBH02116
Class	SE08101	Assessor name	NGO THI MAI LOAN

Plagiarism

Plagiarism is a particular form of cheating. Plagiarism must be avoided at all costs and students who break the rules, however innocently, may be penalised. It is your responsibility to ensure that you understand correct referencing practices. As a university level student, you are expected to use appropriate references throughout and keep carefully detailed notes of all your sources of materials for material you have used in your work, including any material downloaded from the Internet. Please consult the relevant unit lecturer or your course tutor if you need any further advice.

Student Declaration

I certify that the assignment submission is entirely my own work and I fully understand the consequences of plagiarism. I declare that the work submitted for assessment has been carried out without assistance other than that which is acceptable according to the rules of the specification. I certify I have clearly referenced any sources and any artificial intelligence (AI) tools used in the work. I understand that making a false declaration is a form of malpractice.

Student's signature	HUYNH
----------------------------	-------

☐ **Summative Feedback:**☐ **Resubmission Feedback:****Grade:****Assessor Signature:****Date:****Internal Verifier's Comments:****Signature & Date:**

Table of Contents

Introduction.....	7
P1. Describe two iterative and two sequential software lifecycle models.....	7
1. Waterfall SDLC Models.....	8
2. Iterative SDLC Models	10
3. V-models (Verification and Validation Models) in SDLC	12
4. Spiral SDLC Models.....	14
Model suitable for the project Below is a concise breakdown of 4 SDLC models and why the Iterative Model is the premier choice for a Hotel Room Booking System.	15
P2. Explain how risk is managed in software lifecycle models	20
What is risk management in software development?	20
In software development models, which model focuses on risk management?	20
Risk management methods in various models	20
Identify project risks and risk management methods.	22
P3. Explain the purpose of a feasibility report	25
P4. Describe how technical solutions can be compared.....	29
P5. Undertake a software investigation to meet the business need.....	33
P6. Create software investigation and supporting documentation.....	41
Functional and Non-Functional Requirements	41
Use case diagram (include Use case Description).....	44
What is a use case diagram?	44
When to apply use case diagrams.....	44
System use case diagram	46
Explain how to draw an ERD diagram	46
P7. Discuss, using examples, the suitability of software behavioural design techniques	50
Using wireframe to design website.....	50
Design database	55
Show your database diagram.....	57
The specific functionality that the system have includes:	57

M2. Discuss the components of a feasibility report.	66
M3. Analyse how software requirements can be traced throughout the software lifecycle.....	69
M4. Discuss two approaches to improving software quality.....	72
M5. Analyse a range of software behavioural tools and techniques.....	82
M6. Differentiate between a finite state machine (FSM) and an extended FSM, providing an application for both.....	85
References.....	89

Figure 1: SDLC Models.....	7
Figure 2: Waterfall SDLC Models.....	8
Figure 3: Iterative SDLC Models	10
Figure 4: V-models.....	12
Figure 5: Spiral SDLC Models	14
Figure 6: use case diagram	46
Figure 7: ERD diagram	47
Figure 8: Img dashboard.....	50
Figure 9: Wireframe all hotels.....	51
Figure 10: Wireframe book	52
Figure 11: Wireframe login.....	53
Figure 12: Wireframe register	54
Figure 13: Wireframe profile.....	55
Figure 14: Database diagram	57
Figure 15: Landing page login.....	58
Figure 16: Landing page register	59
Figure 17: Landing page dashboard	60
Figure 18: Landing page dashboard	61
Figure 19: Landing page dashboard	62
Figure 20: Landing page admin dashboard	63
Figure 21: Landing page manage hotels.....	63
Figure 22: Landing page rooms	64
Figure 23: Landing page admin bookings.....	64
Figure 24: Landing page admin users.....	65
Figure 25: Landing page admin destinations	65
Figure 26: Landing page admin review	66
Figure 27: Test plan login	74
Figure 28: Test plan error	75
Figure 29: Test plan bookings.....	76
Figure 30: Test plan confirm booking.....	76
Figure 31: Test plan admin dashboard.....	77
Figure 32: Test plan admin manage users.....	77
Figure 33: Test plan admin delete users	78
Figure 34: Test plan admin manage bookings.....	78
Figure 35: Test plan admin manage bookings.....	79
Figure 36: Login flowchart.....	80
Figure 37: Booking flowchart	81
Figure 38: Admin flowchart.....	82

Introduction

You are a **Software Development Consultant** at **InnovateTech Solutions**, a global leader in custom software solutions. Your expertise in planning and implementing software development lifecycles has earned you the responsibility of leading the planning phase for a critical project. Your client has approached you to design and plan a robust web-based software system tailored to their specific needs.

As part of this project, you are required to produce a comprehensive **Software Development Plan (SDP)** that outlines the lifecycle methodology, feasibility analysis, functional and non-functional requirements, and system design strategies. This plan will serve as the foundation for the development process, ensuring that the system meets the client's expectations while adhering to industry standards.

P1. Describe two iterative and two sequential software lifecycle models

SDLC Models or Software Development Life Cycle (SDLC) models are frameworks that guide the development process of software applications from initiation to deployment. Various SDLC models in software engineering exist, each with its approach to the phases of development.



Figure 1: SDLC Models

1. Waterfall SDLC Models

The Waterfall model is one of the oldest and most straightforward approaches to software development. In this blog, we will explore the key aspects of the Waterfall Model, its phases, advantages, disadvantages, and instances where it is most suitable.

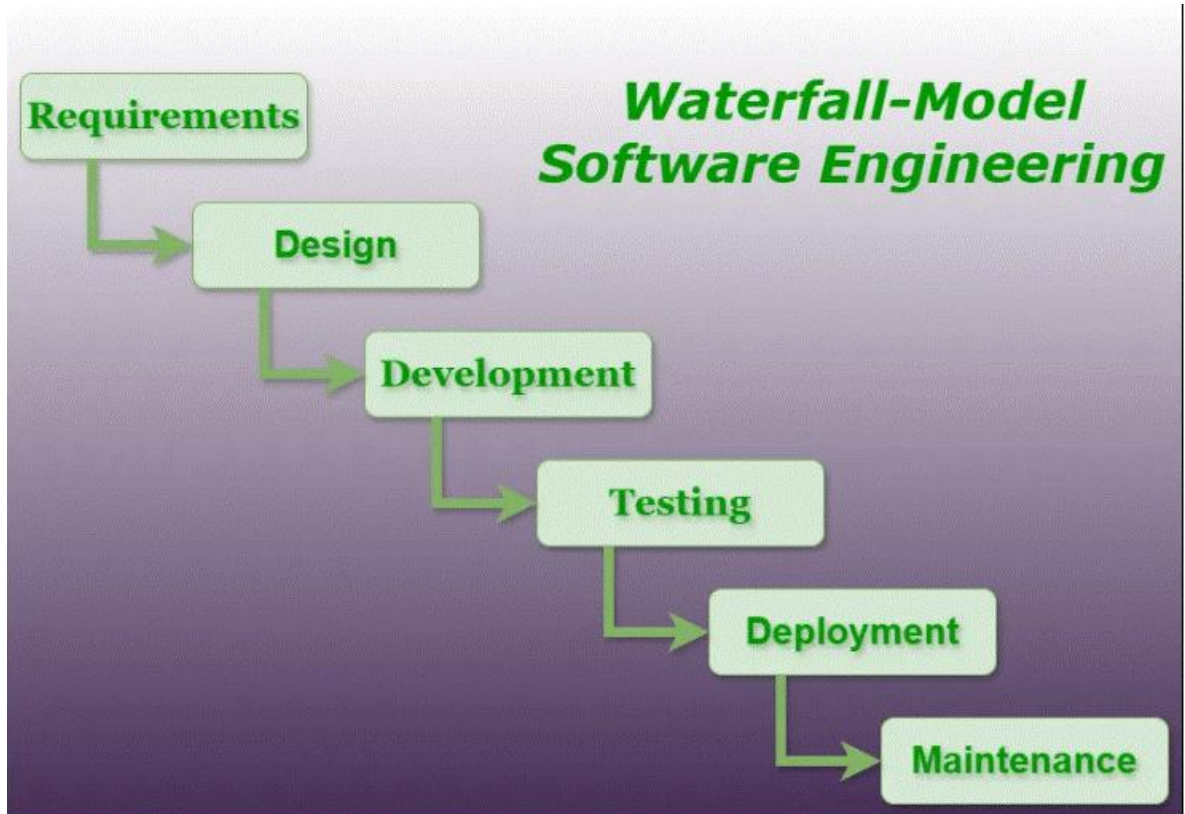


Figure 2: Waterfall SDLC Models

1. **Requirements:** The first phase involves gathering requirements from stakeholders and analyzing them to understand the scope and objectives of the project.
2. **Design:** Once the requirements are understood, the design phase begins. This involves creating a detailed design document that outlines the software architecture, user interface, and system components.
3. **Development:** The Development phase includes implementation involves coding the software based on the design specifications. This phase also includes unit testing to ensure that each component of the software is working as expected.
4. **Testing:** In the testing phase, the software is tested as a whole to ensure that it meets the requirements and is free from defects.

5. **Deployment:** Once the software has been tested and approved, it is deployed to the production environment.
6. **Maintenance:** The final phase of the Waterfall Model is maintenance, which involves fixing any issues that arise after the software has been deployed and ensuring that it continues to meet the requirements over time.

Advantages of the Waterfall SDLC Models

- **Simplicity:** The linear and sequential nature of the Waterfall models makes it easy to understand and implement.
- **Clear Documentation:** Each phase has its own set of documentation, making it easier to track progress and manage the project.
- **Stable Requirements:** Well-suited for projects with stable and well-defined requirements at the beginning.
- **Predictability:** Due to its structured nature, the Waterfall model allows for better predictability in terms of timelines and deliverables.

Disadvantages of the Waterfall SDLC Models

- **Rigidity:** The model is highly inflexible once a phase is completed, making it challenging to accommodate changes.
- **Late Testing:** Testing is performed after the implementation phase, so defects might not be discovered until late in the process.
- **Limited Client Involvement:** Clients are involved mainly in the initial phase, and significant changes cannot be easily accommodated later in the development process.
- **No Prototyping:** The models lack the provision for creating prototypes, which could be a disadvantage in projects where user feedback is crucial.

When to Use the Waterfall SDLC Models

- **Well-Defined Requirements:** When project requirements are clear, stable, and unlikely to change significantly.
- **Small to Medium-Sized Projects:** For smaller projects with straightforward objectives and limited complexity.

- **Mission-Critical Systems:** In scenarios where it is crucial to have a well-documented and predictable development process, especially for mission-critical systems.

2. Iterative SDLC Models

In software development, choosing the right SDLC models is crucial for success. Among the various approaches, the Iterative SDLC model stands out as a flexible and efficient methodology that promotes continuous improvement and adaptability. In this blog post, we will explore the intricacies of the Iterative SDLC models, shedding light on its principles, benefits, and best practices

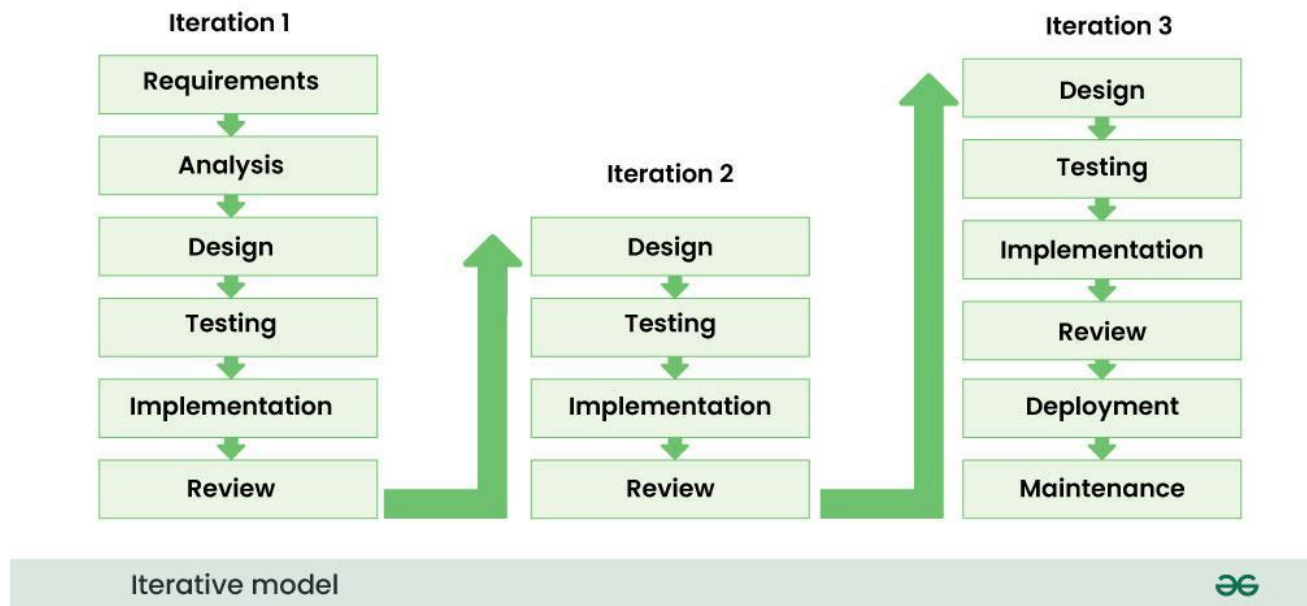


Figure 3: Iterative SDLC Models

Key Principles of Iterative SDLC Models

- **Incremental Progress:** The Iterative model emphasizes incremental development, breaking down the project into manageable parts. This allows for quicker delivery of functional components and facilitates early user feedback.
- **Flexibility and Adaptability:** One of the model's strengths is its adaptability to changing requirements. Developers can easily incorporate new features or modifications during any iteration without disrupting the entire development process.
- **Continuous Evaluation:** Regular assessment and evaluation occur after each iteration, enabling developers to identify and rectify issues early in the development cycle. This continuous feedback loop ensures that the final product aligns with user expectations.

- **Risk Management:** Risks are addressed proactively throughout the development process. By identifying potential issues early on, the team can mitigate risks and make informed decisions, reducing the likelihood of project setbacks.

Benefits of Iterative SDLC Models

- **Faster Time-to-Market:** Incremental development allows for the release of functional components at the end of each iteration, resulting in a faster time-to-market compared to traditional SDLC models.
- **Enhanced Flexibility:** The ability to adapt to changing requirements makes the Iterative models suitable for projects with evolving needs, ensuring that the final product meets user expectations.
- **Improved Quality:** Continuous evaluation and testing in each iteration contribute to higher software quality. Bugs and issues are identified and addressed early, preventing them from accumulating in later stages.
- **Increased Stakeholder Engagement:** Stakeholders are involved throughout the development process, providing valuable feedback after each iteration. This ensures that the final product aligns with user expectations and business goals.

Best Practices for Implementing Iterative SDLC

- **Clear Project Scope:** Define a clear and well-understood project scope to guide each iteration. This ensures that the development team and stakeholders are aligned on the project's goals.
- **Effective Communication:** Open and transparent communication is crucial. Regular team meetings, stakeholder updates, and documentation help maintain a shared understanding of project progress and requirements.
- **Automated Testing:** Implement automated testing to streamline the testing process in each iteration. This ensures that the software remains stable and functional as new features are added.
- **Version Control:** Utilize version control systems to manage code changes and track project history. This helps in maintaining a stable codebase and enables easier collaboration among team members.

3. V-models (Verification and Validation Models) in SDLC

The V-models in SDLC emerge as a compelling alternative, offering a structured and systematic approach. This blog post aims to unravel the intricacies of the V-models SDLC, shedding light on its principles, advantages, and best practices.

The V-Models, also known as the Verification and Validation models, is an extension of the traditional Waterfall models. It introduces a parallel testing phase for each corresponding development stage, forming a V-shaped diagram. Let's delve into the key principles that underpin the V-Models.

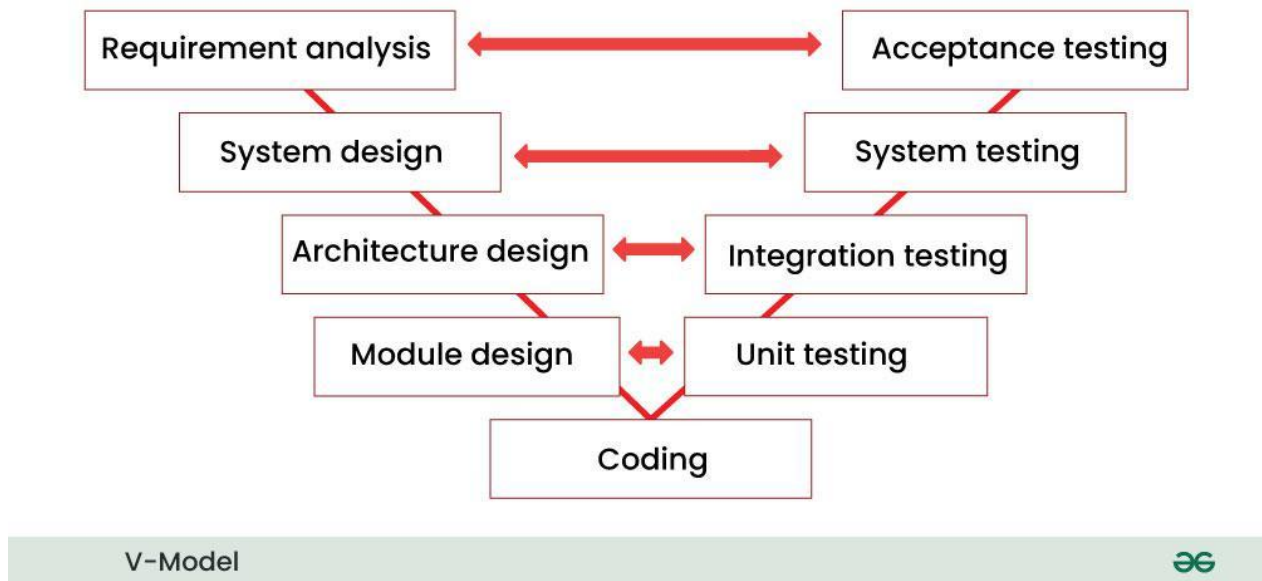


Figure 4: V-models

Key Principles of V-Models

- **Parallel Development and Testing:** Unlike the sequential nature of the Waterfall models, the V-models promote parallel development and testing. Each development phase aligns with a corresponding testing phase, fostering early defect detection.
- **Verification and Validation:** The V-models place equal emphasis on both verification (ensuring that the product is built right) and validation (ensuring that the right product is built). Verification activities align with development phases on the left side of the V, while validation activities correspond to testing phases on the right side.
- **Traceability:** Traceability is a core tenet of the V-Models, ensuring a direct linkage between each development phase and its associated testing phase. This traceability facilitates comprehensive documentation and alignment between requirements, design, and testing activities.

- **Early Defect Detection:** By integrating testing activities in parallel with development, the V-models enable the early detection of defects. This proactive approach contributes to the creation of a more robust and reliable end product.

Advantages of the V-Models

- **Clear Design and Planning:** The V-Models's structured framework facilitates clear design and planning. Well-defined tasks and deliverables at each stage contribute to effective project management.
- **Early Issue Identification:** Incorporating testing early in the development process allows for the timely identification and resolution of issues. This proactive stance minimizes the likelihood of significant defects surfacing later in the project.
- **Traceability and Documentation:** The V-Models's emphasis on traceability ensures a strong connection between development steps and testing steps. This results in thorough documentation, enhancing transparency and aiding project management.
- **Predictability and Control:** The systematic approach of the V-models contributes to predictability and control in the development process. Stakeholders benefit from a clear understanding of each phase, facilitating better management of expectations.

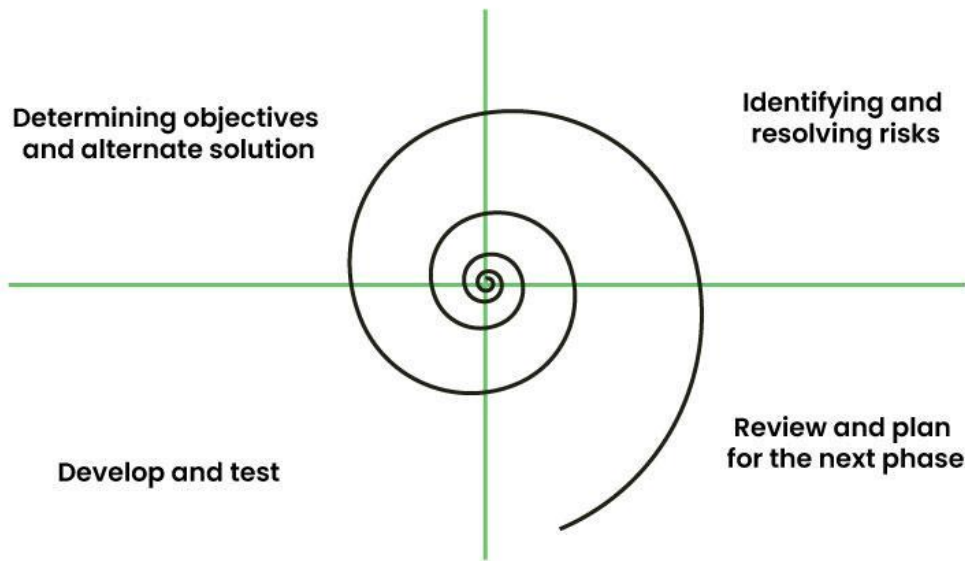
Best Practices for V-models Implementation

- **Thorough Requirements Analysis:** A detailed and well-defined set of requirements is imperative for the success of the V-Models. Thorough requirements analysis ensures alignment with project goals throughout subsequent development and testing activities.
- **Effective Communication:** Clear and consistent communication between development and testing teams is crucial. Regular meetings, status updates, and collaboration tools foster synchronization and enable prompt issue resolution.
- **Automated Testing:** Leveraging automated testing tools enhances the efficiency and effectiveness of the testing process in the V-Models. Automation allows for the seamless execution, repetition, and adaptation of tests as needed.
- **Iterative Feedback Loop:** Establishing an iterative feedback loop between development and testing teams is essential. Insights gained from testing inform and enhance subsequent development phases, fostering continuous improvement.

Disadvantages: Very high cost, requires skilled risk analysts, complex process.

4. Spiral SDLC Models

The Spiral model combines the idea of iterative development with the systematic aspects of the Waterfall model. It is based on the concept of a spiral, with each loop representing a phase in the software development process. The model is inherently risk-driven, meaning that risks are continuously assessed and addressed throughout the development life cycle.



Spiral model



Figure 5: Spiral SDLC Models

Key Principles of Spiral SDLC Models

- **Iterative Development:** The Spiral model embraces iterative development, allowing for the incremental release of the product. Each iteration, or spiral, includes planning, risk analysis, engineering, testing, and evaluation phases.
- **Risk Management:** Risk analysis is a fundamental component of the Spiral Models. Each iteration begins with a risk assessment, and the project progresses based on addressing high-priority risks. This approach allows for proactive risk management and mitigation.
- **Flexibility:** The models are highly adaptable to changes in requirements. As each iteration involves planning and assessment, modifications and adjustments can be made to accommodate evolving project needs.
- **Continuous Evaluation:** Evaluation is integrated into every phase of the Spiral Models. After each iteration, the project is reviewed to assess progress, identify potential risks, and determine the next steps.

Advantages of the Spiral SDLC Models

- **Risk Mitigation:** The focus on risk analysis and management allows for early identification and mitigation of potential issues, reducing the likelihood of project failure.
- **Flexibility in Requirements:** Changes to requirements can be accommodated at any stage of the development process. The iterative nature of the Spiral models facilitates flexibility and adaptation.
- **High-Quality Products:** Continuous evaluation and testing contribute to the production of high-quality software. Defects are identified and addressed early in the development life cycle.
- **Client Involvement:** Clients and stakeholders are involved throughout the development process. Their input is sought during the planning and evaluation phases, ensuring the final product aligns with their expectations.

Best Practices for Spiral SDLC Models

- **Thorough Risk Assessment:** Conduct a comprehensive risk assessment at the beginning of each iteration. Prioritize and address high-risk elements to minimize potential challenges.
- **Regular Review Meetings:** Hold regular review meetings after each iteration to assess progress, evaluate the product, and plan the next steps. This continuous feedback loop is crucial for success.
- **Collaborative Team Communication:** Foster open communication among team members. Collaboration is key to successfully navigating the iterative and dynamic nature of the Spiral Models.
- **Documentation:** Maintain detailed documentation throughout the development process. This includes documentation of risks, decisions, and changes made at each iteration.

(Geeksforgeeks, 2026)

Model suitable for the project

Below is a concise breakdown of 4 SDLC models and why the Iterative Model is the premier choice for a Hotel Room Booking System.

1. Sequential Models (Linear Flow)

A. Waterfall Model

- **Description:** A linear, one-way flow where each phase must be 100% complete before the next begins.

- **Phases:** Requirements → Design → Implementation → Testing → Deployment → Maintenance.
- **Advantages:** Simple to manage; clear milestones; works well for fixed-price contracts.
- **Disadvantages:** High risk; impossible to change requirements mid-way; no working software until the end.
- **When to apply:** Small projects with static, well-defined requirements.

B. V-Model (Validation & Verification)

- **Description:** An extension of Waterfall where every development stage has a corresponding testing stage (forming a "V" shape).
- **Phases:**
 - *Left Side:* Requirement Analysis, System Design, Architecture, Module Design.
 - *Right Side:* Unit Testing, Integration Testing, System Testing, Acceptance Testing.
- **Advantages:** Higher quality due to early test planning; defects are found sooner than Waterfall.
- **Disadvantages:** Rigid; costly to handle changes; no prototype available.
- **When to apply:** Systems requiring high reliability (e.g., medical or aviation software).

2. Iterative Models (Cyclic Flow)

A. Iterative & Incremental Model

- **Description:** The project is broken into small chunks. Each "Iteration" goes through the full SDLC to produce a working version of the software.
- **Phases:** Inception → Iteration (Analysis, Design, Implementation, Testing) → Release.
- **Advantages:** Early feedback; flexible to changes; reduces architecture risks early.
- **Disadvantages:** Needs strong management; "Scope Creep" (project growing too large) is common.
- **When to apply:** Large, complex systems like **E-commerce** or **Booking platforms**.

B. Spiral Model

- **Description:** Combines iterative development with heavy **Risk Analysis**. It moves in loops, focusing on identifying and solving potential failures.

- **Phases:** Identify Objectives → Risk Analysis → Engineering (Build/Test) → Plan Next Phase.
- **Advantages:** Best for risk management; handles complex requirements.
- **Disadvantages:** Very expensive; requires highly skilled risk experts.
- **When to apply:** High-budget, high-risk, mission-critical projects.

A chain of hotels requires a centralized web-based system to streamline their operations and improve customer experience. The system must include:

- **Booking Management:** Allow customers to book rooms online with real-time availability updates.
- **Payment Processing:** Integrate secure payment gateways supporting multiple methods such as credit cards, e-wallets, and bank transfers.
- **Customer Profiles:** Enable customers to create accounts, manage personal information, and view booking history.
- **Reporting Tools:** Generate reports on occupancy rates, revenue trends, and customer demographics.

Comparison Table of SDLC Models

Model	Requirement Response	Suitable or Not	Explanation
Waterfall	Requirements must be clearly defined from the beginning; difficult to change later	Partially suitable	Suitable for small to medium systems with stable requirements. However, hotel booking systems often evolve and require flexibility.
V-Model	Strong verification and validation; testing planned early	Moderately suitable	Ensures high quality but remains rigid like Waterfall and cannot easily accommodate requirement changes.
Iterative & Incremental	Highly flexible; supports continuous improvement and feedback	Most suitable	Allows development in phases (booking, payment, reporting) with early user feedback and gradual system enhancement.

Model	Requirement Response	Suitable or Not	Explanation
Spiral	Focuses heavily on risk analysis at every cycle	Least suitable	Too complex and expensive for a typical hotel booking system; mainly for high-risk projects.
RAD (Rapid Application Development)	Fast development through reusable components and tools	Conditionally suitable	Fast delivery but may weaken long-term architecture and maintainability.
Prototype Model	Focuses on building early versions to clarify requirements	Supportively suitable	Useful for UI and requirement clarification but not ideal as the main development model.

Selecting an appropriate Software Development Life Cycle (SDLC) model is critical to the success of any software project, particularly for complex and customer-oriented systems such as a Hotel Room Booking System. A hotel chain requires a centralized web-based platform to manage online bookings with real-time availability, secure payment processing, customer profile management, and reporting tools. These features are not only complex but also subject to frequent changes due to evolving customer expectations, business strategies, and technological advancements. Therefore, the development model must be flexible, adaptive, and capable of delivering functional software in stages. After evaluating several SDLC models, including Waterfall, V-Model, Spiral, RAD, Prototype, and Iterative & Incremental, the Iterative & Incremental Model emerges as the most suitable choice.

The Waterfall Model follows a linear and sequential approach, where each phase must be completed before moving to the next. Its structure is simple and easy to manage, with clearly defined milestones and documentation. This makes it effective for projects with stable and well-defined requirements. However, a Hotel Room Booking System operates in a dynamic business environment where requirements often evolve. For example, hotels may want to introduce new payment methods, promotional pricing strategies, or mobile-friendly interfaces after development has begun. In the Waterfall model, changes made after the design or implementation stage can be extremely costly and disruptive. Additionally, working software is only delivered at the end of the project, meaning stakeholders cannot interact with the system early to provide feedback. For these reasons, Waterfall is only partially suitable for such a system.

The V-Model improves upon Waterfall by associating each development phase with a corresponding testing phase. This structure ensures high quality because testing is planned early, and defects can be identified sooner. For systems where reliability and correctness are critical, such as medical or aviation software, the V-Model is highly effective. However, it still shares the fundamental limitation of being rigid and sequential. Requirement changes after the design phase can require extensive rework across multiple testing stages. In the context of a hotel booking platform, where features and user expectations may change frequently, this lack of flexibility reduces its practicality. Therefore, the V-Model is considered moderately suitable but not ideal.

The Spiral Model integrates iterative development with intensive risk analysis. Each cycle focuses on identifying potential risks, evaluating solutions, and then developing and testing the system. While this model is powerful for managing uncertainty and complexity, it is also resource-intensive and expensive. It requires experienced risk management professionals and detailed planning for every cycle. A Hotel Room Booking System, although complex, typically does not involve the level of technical or operational risk that justifies such heavy risk management. Using the Spiral Model would significantly increase cost and development time without proportional benefits. Consequently, it is considered the least suitable model for this project.

Rapid Application Development (RAD) emphasizes speed by using automated tools, reusable components, and minimal planning. It can be beneficial when fast delivery is a priority, especially for user interface development or small modules. However, the speed of development can come at the expense of architectural consistency and long-term maintainability. A hotel booking platform must support scalability, security, and integration with payment gateways and databases over time. These requirements demand a well-structured architecture that RAD may not consistently provide. Therefore, RAD can be useful for specific components but is not ideal as the primary development model.

The Prototype Model focuses on building early versions of the system to clarify user requirements. This approach is valuable when stakeholders are unsure of their needs or when user interface design is critical. For example, creating a prototype of the booking interface allows hotel staff and customers to provide feedback before full-scale development begins. However, the Prototype Model primarily addresses requirement discovery rather than providing a comprehensive framework for full system development and maintenance. It is best used as a supporting technique rather than a standalone development methodology.

In contrast, the Iterative & Incremental Model provides the flexibility and structure needed for a Hotel Room Booking System. This model divides the project into smaller functional components, each developed through a complete cycle of analysis, design, implementation, and testing. Each iteration produces a working version of the system that can be evaluated by stakeholders. This allows early detection of problems, continuous user feedback, and gradual improvement of the software.

For example, the first iteration may implement core room management and basic booking functionality. The second iteration could introduce payment processing and transaction security. Subsequent iterations may add customer profile management, reporting tools, and advanced features such as promotions or analytics. This step-by-step approach ensures that the most important features are delivered early while allowing adjustments based on real-world usage.

Another major advantage of the Iterative Model is its adaptability to change. The hospitality industry is highly competitive and constantly evolving. Customer expectations regarding convenience, payment methods, and personalization continue to grow. With iterative development, new features or improvements can be added in future cycles without disrupting the entire system. This ensures that the platform remains relevant and responsive to business needs.

Additionally, iterative development reduces risk by validating system architecture and functionality early. Instead of waiting until the end of the project, developers and stakeholders can continuously verify whether the system meets performance, usability, and business requirements. This leads to better decision-making and higher overall quality.

In conclusion, after evaluating multiple SDLC models, the Iterative & Incremental Model is the most suitable choice for developing a Hotel Room Booking System. Its flexibility, risk reduction, and ability to deliver functional software in stages make it ideal for systems with evolving requirements and multiple interconnected features. By enabling continuous feedback and gradual enhancement, this model supports both technical reliability and business adaptability, ensuring long-term success for the hotel's digital operations.

P2. Explain how risk is managed in software lifecycle models

What is risk management in software development?

Risk management in software engineering is the process of identifying, assessing, mitigating, and monitoring potential risks or uncertainties that could impact the successful completion or quality of a software development project. It involves systematically identifying potential risks, evaluating their likelihood and impact, and taking proactive measures to minimize their negative effects. (CMCglobal, 2023)

In software development models, which model focuses on risk management?

The Spiral model is often used for long projects that have high risk as the model highly focuses on risk management. For that reason, the development team who handles such a project needs to have skills and knowledge in risk management.

Risk management methods in various models

1. Waterfall Model – Risk Management Through Upfront Planning

How risk is managed:

- Risks are primarily identified and analyzed during the early stages, especially in requirements analysis and system design.
- Detailed documentation and structured planning are used to minimize misunderstandings and technical issues.
- Testing is conducted near the end of development, where defects are identified and corrected.
- If major problems occur late in the process, changes are expensive and difficult because earlier phases are already completed.

Key characteristics:

- Focuses on preventing risks through careful planning.
- Works best when requirements are stable and well understood.
- Risks are often discovered late, making them more costly to fix.
- Limited flexibility if requirements change during development.

Overall strategy: Preventive planning and strict process control.

2. Agile Model – Continuous and Adaptive Risk Management**How risk is managed:**

- Development is divided into short iterations or sprints, allowing frequent evaluation of progress.
- Continuous customer feedback helps identify misunderstandings or changing needs early.
- High-priority features are developed first, reducing the risk of delivering low-value functionality.
- Daily team meetings help detect technical or process issues quickly.
- Continuous testing and integration reduce the risk of large-scale system failures.

Key characteristics:

- Risks are identified and addressed continuously throughout the project.
- Highly adaptable to changing requirements.
- Reduces the risk of building a product that does not meet user expectations.

- Requires strong team collaboration and experience to manage effectively.

Overall strategy: Ongoing monitoring, rapid feedback, and flexible response to change.

3. Iterative / Incremental Model – Risk Reduction in Stages

How risk is managed:

- The system is divided into smaller components or increments.
- Each iteration includes analysis, design, development, and testing.
- Problems are identified early when the system is still small and manageable.
- Lessons learned from previous iterations are used to improve future development.
- System architecture can evolve gradually, reducing the risk of large design failures.

Key characteristics:

- Risk is reduced progressively as the system grows.
- Provides working software early for evaluation.
- Supports gradual improvement and controlled change.
- Requires strong management to prevent uncontrolled scope expansion.

Overall strategy: Step-by-step risk reduction through repeated development cycles.

Identify project risks and risk management methods.

1. Technical Risks

Risks

- Integration failures with online payment gateways.
- Inaccurate or delayed real-time room availability updates.
- Poor system performance under high user traffic.
- Security vulnerabilities leading to data breaches.
- System failures during deployment or updates.

Risk Management Methods

- Design a scalable and reliable system architecture.
- Perform regular integration testing and performance testing.
- Apply strong data encryption and security standards.
- Use continuous testing and monitoring after deployment.
- Deploy updates in a staging environment before production release.

2. Requirement Risks

Risks

- Changing requirements during development.
- Unclear or incomplete customer specifications.
- Misunderstanding hotel business processes.
- Scope creep caused by adding unplanned features.

Risk Management Methods

- Use prototypes or mockups to clarify requirements early.
- Document requirements clearly and obtain formal approval.
- Implement a structured change management process.
- Use iterative or Agile development to adapt to change.
- Conduct regular meetings with stakeholders.

3. Project Management Risks

Risks

- Project delays due to poor scheduling.
- Inaccurate cost estimation.
- Lack of resources or unexpected staff turnover.
- Communication gaps between development teams.

Risk Management Methods

- Develop realistic project timelines and schedules.
- Track progress using milestones, sprint reviews, or status reports.
- Define clear roles and responsibilities.
- Use project management tools for task tracking.
- Prepare contingency plans for staffing issues.

4. Operational Risks

Risks

- Users struggling to adopt the new system.
- Data entry errors by hotel staff.
- System downtime affecting hotel operations.
- Difficulties in maintaining or upgrading the system.

Risk Management Methods

- Provide user training and clear documentation.
- Design a user-friendly interface.
- Implement backup systems and disaster recovery plans.
- Perform regular system maintenance and updates.

5. External Risks

Risks

- Changes in legal or regulatory requirements related to payments or data protection.
- Service outages from third-party payment providers.
- Cyberattacks or unauthorized access attempts.
- Infrastructure failures such as server, network, or power outages.

Risk Management Methods

- Comply with relevant data protection and financial regulations.

- Provide alternative payment options.
- Use firewalls and intrusion detection systems.
- Host the system on reliable cloud infrastructure with redundancy.

6. Risk Management Process

An effective risk management strategy typically follows these steps:

1. **Risk Identification** – Identify all potential project risks.
2. **Risk Analysis** – Evaluate likelihood and impact.
3. **Risk Prioritization** – Rank risks based on severity.
4. **Risk Mitigation Planning** – Develop strategies to prevent or reduce risk.
5. **Risk Monitoring and Control** – Continuously track and manage risks throughout the project.

P3. Explain the purpose of a feasibility report

1. What is Project Feasibility?

Project feasibility refers to the process of evaluating whether a proposed project is practical, achievable, and worth implementing before significant time, money, and resources are invested. It examines whether the project can be successfully completed given technical capabilities, financial resources, and organizational conditions.

In software development, feasibility analysis helps determine if the system can be built, operated, and maintained effectively while delivering expected benefits to stakeholders.

A project is considered feasible if it can be implemented within acceptable cost, time, and resource limits while meeting business objectives.

2. Purpose of Conducting a Feasibility Study

A feasibility study is conducted before the actual development begins to ensure that the project is realistic and viable.

The main purposes are:

- To evaluate whether the project can be successfully implemented.
- To identify potential risks, limitations, and constraints early.

- To determine required resources such as budget, technology, and personnel.
- To support informed decision-making by stakeholders.
- To avoid wasting time and money on impractical projects.
- To compare alternative solutions and select the best one.

In short, the feasibility study acts as a decision-making tool that helps organizations decide whether to proceed, modify, or cancel a project.

3. Benefits of Conducting a Feasibility Study

Conducting a feasibility study provides several important advantages:

Better Decision Making

It provides objective information that helps managers decide whether the project should move forward.

Risk Reduction

Potential technical, financial, or operational problems are identified early, reducing the chance of project failure.

Cost Control

It estimates development and operational costs, helping organizations plan budgets accurately.

Efficient Resource Allocation

Organizations can determine whether they have sufficient human resources, technology, and infrastructure.

Clear Project Direction

It clarifies project scope, goals, and expectations before development begins.

Stakeholder Confidence

A well-prepared feasibility report increases confidence among investors, managers, and users.

4. Components of a Feasibility Report

A feasibility report typically evaluates three major aspects:

1. Economic feasibility

2. Technical feasibility
3. Organizational feasibility

Each aspect examines a different dimension of project viability.

5. Feasibility Analysis Applied to the Hotel Room Booking System Scenario

The scenario describes a centralized web-based system for a hotel chain that includes:

- Online room booking with real-time availability
- Secure payment processing
- Customer account management
- Reporting tools

Below is a detailed feasibility analysis.

5.1 Economic Feasibility

Economic feasibility evaluates whether the financial benefits of the system justify its costs.

Costs

- Software development cost (developers, designers, testers)
- Infrastructure cost (servers, cloud hosting, databases)
- Payment gateway integration fees
- Security implementation
- Maintenance and technical support
- Staff training
- System upgrades

Benefits

- Increased online bookings
- Reduced manual administrative work
- Improved customer experience

- Better business decision-making through reports
- Higher operational efficiency
- Reduced human errors
- Competitive advantage in the market

Cost–Benefit Evaluation

Although initial development and setup costs may be high, the long-term financial benefits such as increased revenue, reduced operational costs, and improved efficiency outweigh the expenses.

Conclusion (Economic): Economically feasible.

5.2 Technical Feasibility

Technical feasibility examines whether the required technology, tools, and expertise are available.

Technical Requirements

- Web development frameworks
- Secure payment integration
- Database management systems
- Real-time data synchronization
- Cybersecurity mechanisms
- Scalable cloud infrastructure

Technology Availability

Modern technologies such as cloud computing, web frameworks, and payment APIs are widely available and reliable.

Technical Expertise

Software developers with experience in web systems, database management, and security are commonly available in the industry.

System Complexity

Although the system is moderately complex, it is technically achievable using standard development tools and architectures.

Conclusion (Technical): Technically feasible.

5.3 Organizational Feasibility

Organizational feasibility evaluates whether the system fits within the organization's structure, culture, and operational capabilities.

Management Support

A centralized booking system aligns with business goals of improving efficiency and customer satisfaction.

User Acceptance

Hotel staff and customers are familiar with digital booking platforms, increasing the likelihood of acceptance.

Operational Integration

The system can integrate with existing hotel operations such as reservations, billing, and reporting.

Training Requirements

Staff training is manageable and not overly complex.

Organizational Readiness

Hotels benefit from automation and centralized management, making implementation practical.

Conclusion (Organizational): Organizationally feasible.

P4. Describe how technical solutions can be compared

1. Comparing Technical Solutions

When developing a Hotel Room Booking System, different technical solutions can be considered. Each solution must be evaluated based on feasibility criteria such as technical capability, cost, and organizational readiness. Comparing technical alternatives helps decision-makers choose the most practical and beneficial solution for the organization.

The three main technical approaches are:

1. Developing the system fully in-house (custom development)

2. Purchasing a complete external system
3. Purchasing some modules and developing the rest (hybrid approach)

2. Possible Technical Solutions and Comparison

Solution	Advantages	Disadvantages
Fully custom development (build web system in-house)	Easy to modify and update; full control over system features; flexible and scalable; tailored to hotel business processes	Higher development time; requires skilled developers; full responsibility for maintenance and security
Fully purchased external system	Fast implementation; professional interface; vendor support and maintenance; proven reliability	Limited customization; expensive licensing fees; difficult to update or change features; dependency on vendor
Purchase some modules (hybrid solution)	Balanced cost and flexibility; faster than full development; access to reliable third-party components (e.g., payment gateway)	Integration complexity; compatibility issues; partial vendor dependency; requires technical expertise

3. Evaluation Based on the Scenario

The hotel booking system requires:

- Real-time booking management
- Secure online payment
- Customer accounts
- Reporting tools

A fully purchased system may not fully match business requirements and may involve high long-term costs. A hybrid approach offers flexibility but requires integration management. Full custom development provides maximum control and customization, which is important for long-term scalability and business growth.

Therefore, custom development is considered the most suitable general approach.

4. Weighted Decision Matrix for Technical Implementation

To determine the best development technology, three technical implementation options are compared:

- ASP.NET
- PHP, HTML, CSS, JavaScript
- Java

Feasibility Criteria and Weights

- Technical feasibility: 30%
- Economic feasibility: 40%
- Organizational feasibility: 30%

5. Weighted Scoring Matrix

Feasibility Criteria	Weight	ASP.NET	PHP, HTML, CSS, JS	Java
Technical feasibility	30%	10 (team lacks experience)	80 (team has experience)	10 (team lacks experience)
Economic feasibility	40%	Cost: \$2500 → Score 60	Cost: \$1500 → Score 90	Cost: \$2000 → Score 70
Organizational feasibility	30%	70	70	70

6. Weighted Score Calculation

To select the most suitable technical solution, the team uses a **Weighted Decision Matrix**.

This method evaluates each solution based on key feasibility criteria and assigns a weighted score according to the importance of each criterion.

6.1 Calculation Formula

Each evaluation criterion includes:

- **Weight (%)** → importance level of the criterion

- **Score (0–100)** → performance of each solution

The weighted score is calculated as:

$$\text{Weighted Score} = \text{Score} \times \text{Weight}$$

The total score of a solution is the sum of all weighted scores.

6.2 Evaluation Criteria and Weights

Feasibility Criteria	Weight
Technical feasibility	30%
Economical feasibility	40%
Organizational feasibility	30%
Total	100%

6.3 Scores Assigned to Each Technical Solution

Criteria	ASP.NET	PHP + HTML + CSS + JS	Java
Technical feasibility	10	80	10
Economical feasibility	60	90	70
Organizational feasibility	70	70	70

6.4 Weighted Score Calculation Table

Criteria	Weight	ASP.NET Score	ASP.NET Weighted	PHP Score	PHP Weighted	Java Score	Java Weighted
Technical feasibility	30%	10	$10 \times 0.30 = 3$	80	$80 \times 0.30 = 24$	10	$10 \times 0.30 = 3$

Criteria	Weight	ASP.NET Score	ASP.NET Weighted	PHP Score	PHP Weighted	Java Score	Java Weighted
Economical feasibility	40%	60	$60 \times 0.40 = 24$	90	$90 \times 0.40 = 36$	70	$70 \times 0.40 = 28$
Organizational feasibility	30%	70	$70 \times 0.30 = 21$	70	$70 \times 0.30 = 21$	70	$70 \times 0.30 = 21$
Total Score	100%		48		81		52

6.5 Solution Ranking

Solution	Total Weighted Score	Rank
PHP + HTML + CSS + JS	81	1
Java	52	2
ASP.NET	48	3

6.6 Result Analysis

The **PHP + HTML + CSS + JavaScript** solution achieves the highest score (81). This indicates that it performs best across the evaluated criteria, especially in:

- Technical feasibility (team has strong experience)
- Economic feasibility (lowest development cost)
- Organizational feasibility (suitable for team structure and resources)

P5. Undertake a software investigation to meet the business need

1. Techniques Used to Gather Requirements

Different requirement-gathering techniques are used in software investigation. Each method provides different types of information and is suitable for different situations.

1.1 Description of Software Investigation Techniques

1. Interview

A direct communication method where stakeholders are asked questions to understand their needs, expectations, and problems with the current system.

Advantages

- Provides detailed and accurate information
- Allows clarification and follow-up questions
- Helps understand user opinions and experiences

Disadvantages

- Time-consuming
- Requires preparation and scheduling
- Results may be subjective

2. JAD (Joint Application Development) Sessions

Structured group workshops involving developers, managers, and users to define requirements collaboratively.

Advantages

- Faster decision-making
- Encourages collaboration
- Reduces misunderstandings

Disadvantages

- Requires skilled facilitation
- Difficult to coordinate participants
- May be costly

3. Questionnaires

A written set of questions distributed to many users to collect data quickly.

Advantages

- Collects information from many people
- Low cost
- Easy to analyze statistically

Disadvantages

- Limited detail
- No immediate clarification
- Risk of low response rate

4. Documentation Analysis

Reviewing existing documents such as reports, manuals, procedures, and system records.

Advantages

- Provides historical and factual information
- Helps understand current processes
- No interruption to users

Disadvantages

- Documents may be outdated
- May not show real user problems

5. Observation

Watching users perform their daily tasks to understand real workflows.

Advantages

- Shows actual system usage
- Identifies hidden problems
- Reveals inefficiencies

Disadvantages

- Time-consuming
- Users may change behavior when observed

1.2 Choosing an Appropriate Method for the Scenario

For this project, **interviews** are selected as the primary investigation method.

Reason for selection:

- The system involves multiple stakeholders with different roles
- Management decisions require strategic input
- Technical requirements must be clearly defined
- User experience and satisfaction must be understood

Interviews provide deep, detailed, and role-specific information that other methods cannot easily capture.

1.3 Undertaking the Investigation

The software investigation using interviews includes four main steps.

Step 1 — Select Interview Participants

Key stakeholders from different roles are chosen to ensure complete requirement coverage.

Full Name	Role	Interview Purpose	Date	Time
Nguyễn Văn A	Chief Executive Officer	Understand strategic goals and business vision	10 Mar 2026	09:00
Nguyễn Văn B	Chief Technology Officer	Identify required technologies and system architecture	10 Mar 2026	10:30
Nguyễn Văn C	Customer	Evaluate usability and service expectations	11 Mar 2026	09:00
Nguyễn Văn D	Seller	Understand operational workflow and sales process	11 Mar 2026	11:00

Full Name	Role	Interview Purpose	Date	Time
Nguyễn Văn E	IT Staff	Identify maintenance, security, and technical constraints	12 Mar 2026	09:30

Step 2 — Design Interview Questions

Different types of questions are prepared to collect comprehensive information.

Closed-Ended Questions

Used to obtain factual or yes/no information.

Examples:

- Are you currently using the existing digital platform system?
- Do you experience system downtime regularly?
- Is the current system secure enough?

Open-Ended Questions

Used to explore opinions, expectations, and improvement suggestions.

Examples:

- What do you think about the current system?
- Has the system met operational requirements?
- What improvements would you recommend?
- How should user roles be managed?
- What system performance do you expect?
- What are your security requirements?
- What availability level is required?
- Are there any installation or configuration requirements?

Probing Questions

Used to obtain deeper clarification.

Examples:

- Can you explain your experience with the current system?
- What specific features should be changed?
- Why is this issue important for your work?

Step 3 — Conduct the Interviews (Record the Interview Process)

The interviews were conducted over three working days with key stakeholders from different departments. Each interview lasted approximately 30–45 minutes and followed a semi-structured format, allowing both prepared and follow-up questions.

Interview Procedure

1. The interviewer introduced the purpose of the study and explained how the information would be used.
2. Consent was obtained to take written notes during the discussion.
3. Questions were asked according to the prepared interview guide.
4. Additional probing questions were used to clarify responses.
5. The interviewer summarized the key points at the end to confirm accuracy.

Interview Records

Interview 1 — Chief Executive Officer

- **Focus:** Business strategy and organizational goals
- **Key Points:**
 - The organization needs a centralized web-based system to improve operational efficiency.
 - Real-time reporting is essential for decision-making.
 - The system must support business growth and future expansion.
 - Customer satisfaction and service speed are top priorities.

Interview 2 — Chief Technology Officer

- **Focus:** Technical requirements and infrastructure
- **Key Points:**
 - The system must integrate with existing databases and services.
 - Security is a major concern, especially payment protection.
 - The system should be scalable and support high user traffic.
 - Regular backup and recovery mechanisms are required.

Interview 3 — Customers

- **Focus:** User experience and service expectations
- **Key Points:**
 - Customers want a simple and fast booking process.
 - They expect real-time availability updates.
 - Secure online payment options are required.
 - Access to booking history and account management is important.

Interview 4 — Sellers / Operational Staff

- **Focus:** Daily operations and workflow
- **Key Points:**
 - Manual processes currently cause delays and errors.
 - Automated booking and confirmation features are needed.
 - The system should generate reports for monitoring performance.
 - Easy search and filtering functions are required.

Interview 5 — IT Staff

- **Focus:** System maintenance and technical support
- **Key Points:**
 - The system must be easy to maintain and update.

- Monitoring tools are needed to detect system issues.
- Role-based access control is required for security.
- Clear installation and configuration procedures are necessary.

Step 4 — Create Interview Reports

After completing the interviews, the collected data was analyzed and organized into requirement categories.

1. Business Requirements

- Improve operational efficiency
- Enhance customer experience
- Support business growth and scalability
- Provide accurate and real-time reporting

2. Functional Requirements

- Online booking management
- User account and profile management
- Secure payment processing
- Role-based access control
- Reporting and analytics tools
- Search and filtering functions

3. Non-Functional Requirements

- High system availability
- Fast response time
- Strong data security and privacy protection
- System scalability

- Easy maintenance and monitoring

4. Technical Requirements

- Integration with existing systems
- Database management
- Backup and recovery mechanisms
- Secure authentication and authorization
- Web-based architecture

5. Identified Problems in Current Operations

- Manual processes cause delays and errors
- Limited reporting capability
- Poor system integration
- Lack of automation
- Security risks in data handling

6. Stakeholder Expectations

- Reliable and secure system
- Easy-to-use interface
- Real-time data updates
- Automated processes
- Efficient system management

P6. Create software investigation and supporting documentation

Functional and Non-Functional Requirements

Based on the software investigation and stakeholder interviews, the system requirements are categorized into **Functional Requirements** and **Non-Functional Requirements**.

Functional requirements describe **what the system must do**, while non-functional requirements describe **how the system must perform**.

A. Functional Requirements

These define the main features and operations of the system.

No.	Functional Requirement	Description
FR1	User Registration	Customers must be able to create accounts using personal information.
FR2	User Login & Authentication	Users must log in securely using username and password.
FR3	Role-Based Access Control	Different access permissions for admin, staff, and customers.
FR4	Room Search	Users can search available rooms by date, price, and type.
FR5	Real-Time Availability	The system must show real-time room availability.
FR6	Booking Management	Customers can create, modify, and cancel bookings.
FR7	Payment Processing	The system supports secure online payments (credit card, e-wallet, bank transfer).
FR8	Booking Confirmation	Automatic confirmation after successful booking or payment.
FR9	Customer Profile Management	Users can update personal information and view booking history.
FR10	Reporting and Analytics	Admin can generate reports on occupancy, revenue, and customer data.
FR11	Notification System	System sends booking confirmation and reminders.
FR12	Admin Management	Admin can manage rooms, pricing, and system settings.

B. Non-Functional Requirements

These define system quality, performance, and operational standards.

1. Performance Requirements

- The system must respond to user actions within 2–3 seconds.
- Support multiple users accessing the system simultaneously.
- Handle peak booking periods without performance degradation.

2. Security Requirements

- Secure login authentication.
- Data encryption for payment transactions.
- Role-based access control.
- Protection against unauthorized access.
- Regular system backups.

3. Availability Requirements

- System uptime must be at least 99%.
- Accessible 24/7 via web browser.
- Automatic recovery after system failure.

4. Usability Requirements

- User-friendly interface.
- Simple navigation and booking process.
- Mobile and desktop compatibility.

5. Scalability Requirements

- System must support business growth.
- Ability to handle increasing number of users and bookings.

6. Maintainability Requirements

- Easy system updates and bug fixes.

- Modular architecture for future feature expansion.
- Monitoring tools for system performance.

7. Reliability Requirements

- Accurate booking and payment processing.
- No data loss during system operation.
- Consistent system performance.

8. Compatibility Requirements

- Works on major web browsers.
- Integrates with external payment gateways.
- Compatible with existing database systems.

Use case diagram (include Use case Description)

What is a use case diagram?

In the Unified Modeling Language (UML), a use case diagram can summarize the details of your system's users (also known as actors) and their interactions with the system. To build one, you'll use a set of specialized symbols and connectors. An effective use case diagram can help your team discuss and represent:

- **Scenarios in which your system or application interacts with people, organizations, or external systems**
- **Goals that your system or application helps those entities (known as actors) achieve**
- **The scope of your system**

When to apply use case diagrams

A use case diagram doesn't go into a lot of detail—for example, don't expect it to model the order in which steps are performed. Instead, a proper use case diagram depicts a high-level overview of the relationship between use cases, actors, and systems. Experts recommend that use case diagrams be used to supplement a more descriptive textual use case.

UML is the modeling toolkit that you can use to build your diagrams. Use cases are represented with a labeled oval shape. Stick figures represent actors in the process, and the actor's participation in the system is modeled with a line between the actor and use case. To depict the system boundary, draw a box around the use case itself.

UML use case diagrams are ideal for:

- Representing the goals of system-user interactions
- Defining and organizing functional requirements in a system
- Specifying the context and requirements of a system
- Modeling the basic flow of events in a use case

UML use case diagrams are ideal for:

- Representing the goals of system-user interactions
- Defining and organizing functional requirements in a system
- Specifying the context and requirements of a system
- Modeling the basic flow of events in a use case. (Lucidchart, 2026)

System use case diagram



Figure 6: use case diagram

Explain how to draw an ERD diagram

An entity relationship diagram (ERD) is a visual representation that shows how different pieces of information in a database relate to each other. Think of it as a blueprint for your data structure. It displays entities (like “Customer” or “Order”), their attributes (like “email address” or “order date”), and the relationships between them (like “a customer places many orders”). ERDs bridge the gap between business requirements and technical implementation, giving both developers and stakeholders a clear picture of how data flows through a system.

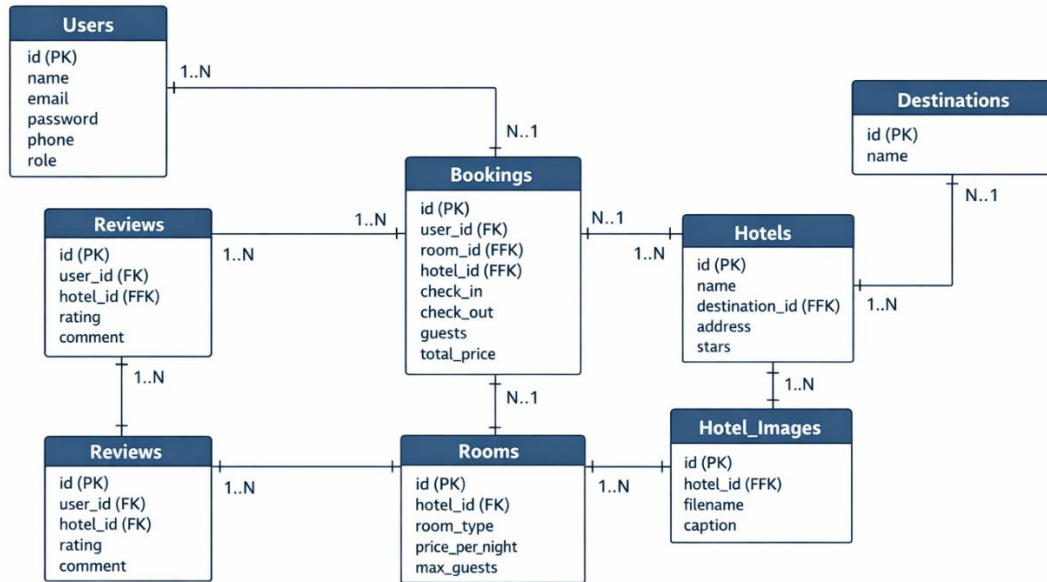


Figure 7: ERD diagram

When and why to use ERDs

se ERDs whenever you're working with structured data that needs to persist beyond a single session:

Starting a new project: Before writing database schemas or API contracts, map out your data model with an ERD. This prevents the painful refactoring that happens when you realize halfway through development that your structure won't support key features.

Planning major features: Adding multi-tenancy to your SaaS product? Implementing a recommendation engine? These features often require significant changes to your data model. ERDs let you test different structural approaches before committing to one.

Onboarding new team members: A well-documented ERD cuts onboarding time dramatically. New engineers can see the entire data architecture at a glance instead of piecing it together from table definitions and join queries.

Communicating with stakeholders: Non-technical stakeholders may not understand database schemas, but they can grasp an ERD. This makes design reviews and requirement validation much more productive.

Key Components of an ERD

Entities

Entities are the core concepts your system tracks. They are the “things” that have data associated with them. In an e-commerce system, entities include Customer, Product, Order, and Category. In a library system, you’d have Book, Member, and Loan.

Think of entities as nouns that represent distinct, independent concepts requiring multiple pieces of information to describe them. Each entity becomes a table in your database. Entities are typically drawn as rectangles with the entity name at the top.

Naming convention: Use singular nouns (Customer, not Customers) because each entity represents one instance of that concept.

Attributes

Attributes are the individual pieces of data that describe an entity. For a Customer entity, attributes might include `customer_id`, `first_name`, `last_name`, `email`, and `phone_number`. These attributes become columns in your database table.

Every entity needs a **primary key, which is** a special attribute that uniquely identifies each instance. The primary key ensures you can distinguish between two customers even if they have the same name. Primary keys are typically marked with “PK” or underlined in ERD diagrams.

Attributes are listed inside the entity rectangle, below the entity name.

Relationships

Relationships define how entities connect to each other. They answer questions like “How do customers relate to orders?” or “What’s the connection between products and categories?”

Relationships are represented by lines connecting entities, often with a verb describing the connection:

- Customer **places** Order
- Product **belongs to** Category
- Employee **manages** Employee

In your database, relationships are implemented through foreign keys. These are attributes in one table that reference the primary key of another table.

Cardinality

Cardinality specifies how many instances of one entity can relate to instances of another entity. It is shown using symbols at the ends of relationship lines.

One-to-one (1:1): Each instance of Entity A relates to exactly one instance of Entity B. Example: Each Employee has one CompanyLaptop. Shown with “1” on both ends or single vertical lines.

One-to-many (1:N): Each instance of Entity A can relate to multiple instances of Entity B, but each B relates to only one A. Example: One Customer places many Orders. Shown with “1” on one end and a crow’s foot (three-pronged fork) or “N” on the other.

Many-to-many (M:N): Instances of both entities can relate to multiple instances of the other. Example: Students enroll in multiple Courses, and Courses have multiple Students. Shown with crow’s feet or “N” on both ends. These require a junction table to implement.

Understanding these four components—entities, attributes, relationships, and cardinality—gives you the vocabulary to read, create, and discuss entity relationship diagrams effectively. (Miro, 2026)

P7. Discuss, using examples, the suitability of software behavioural design techniques

Using wireframe to design website

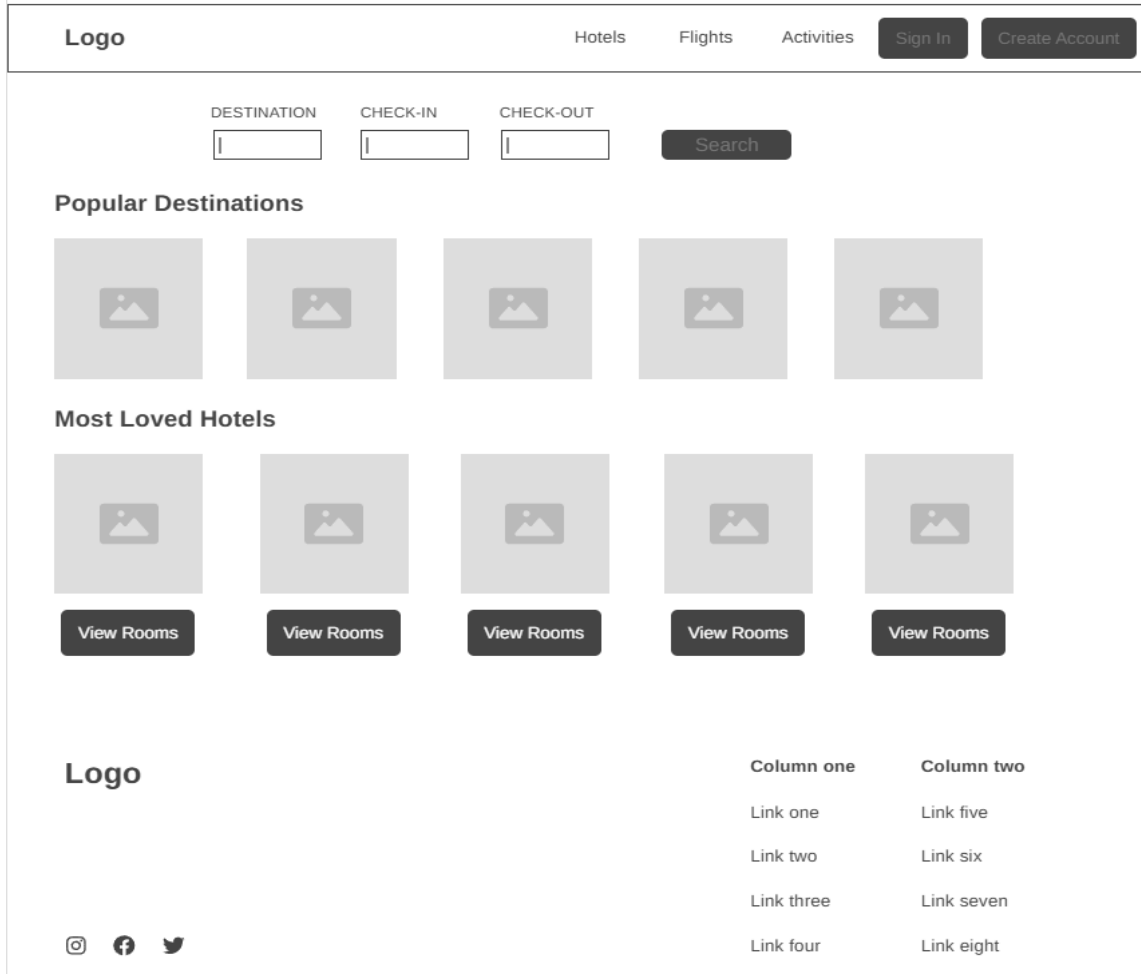


Figure 8: Img dashboard

DESTINATION

CHECK-IN

CHECK-OUT

All Hotels

Logo



Column one

Link one

Link two

Link three

Link four

Column two

Link five

Link six

Link seven

Link eight

Figure 9: Wireframe all hotels

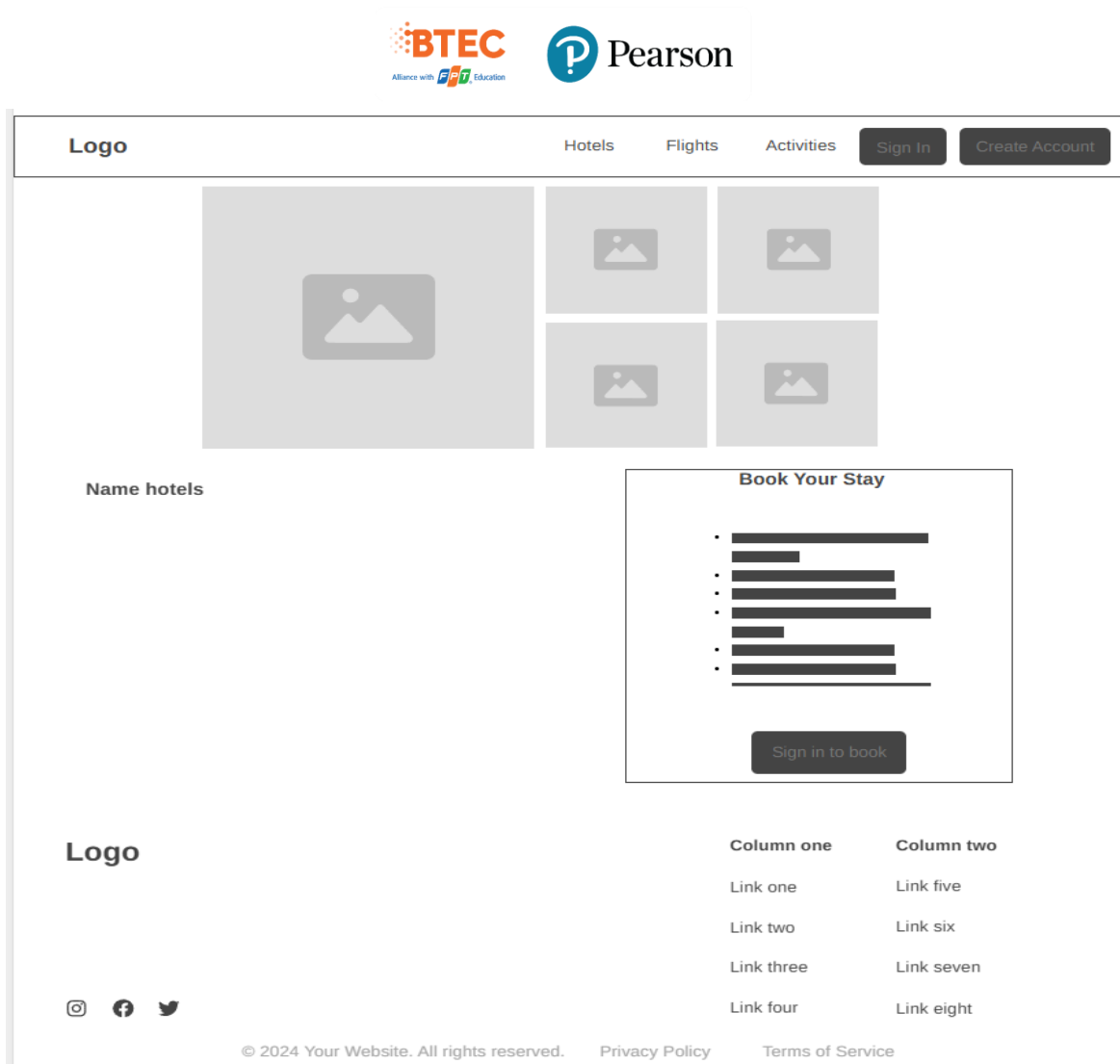


Figure 10: Wireframe book

Sign up form

Username

Email

Password

Sign up

Don't have an account? **Sign up now**

Logo



© 2024 Your Website. All rights reserved.

[Privacy Policy](#)

Terms of Service

Column one

Column two

Link one

Link five

Link two

Link six

Link three

Link seven

Link four

Link eight

Figure 11: Wireframe login

[Create a new account](#)

Username

Email

Phone number

Password

Confirm Password

Create Account

Already have an account? **Sign in**

Logo

Column one

Column two

Link one

Link five

Link two

Link six

Link three

Link seven

Link four

Link eight



Figure 12: Wireframe register

The wireframe shows a user profile page. At the top is a navigation bar with a 'Logo' placeholder, links for 'Hotels', 'Flights', and 'Activities', and buttons for 'Sign In' and 'Create Account'. The main content area is titled 'My Profile' and contains a form with the following fields: 'Full Name', 'Email Address', 'Phone number', 'Change Password', 'New Password', and 'Confirm New Password'. Below the form is a 'Button'. The footer includes a 'Logo' placeholder, social media icons for Instagram, Facebook, and Twitter, and a table with two columns: 'Column one' and 'Column two'. The table contains links labeled 'Link one' through 'Link eight'. At the bottom of the footer is a copyright notice '© 2024 Your Website. All rights reserved.' and links to 'Privacy Policy' and 'Terms of Service'.

Figure 13: Wireframe profile

Design database

Convert ERD to Relation Diagram

1. Relational Schema

In this notation, **Bold** denotes a Primary Key (PK) and *Italics* denotes a Foreign Key (FK).

- **destinations** (**id**, name, country, description, image, created_at)
- **hotels** (**id**, name, *destination_id*, address, description, stars, image, amenities, is_active, created_at)
 - *FK: destination_id → destinations(id)*
- **hotel_images** (**id**, *hotel_id*, filename, caption, sort_order, created_at)

- *FK: hotel_id → hotels(id)*
- bookings (id, user_id, room_id, hotel_id, check_in, check_out, guests, total_price, status, special_requests, created_at)
 - *FK: hotel_id → hotels(id)*
 - *FK: user_id → users(id)* (Implicit)
 - *FK: room_id → rooms(id)* (Implicit)

2. Relationship Analysis

Relationship	Type	Description
Destinations to Hotels	1:N	One destination can have multiple hotels, but a hotel belongs to only one destination.
Hotels to Hotel Images	1:N	A hotel can have a gallery of many images.
Hotels to Bookings	1:N	A hotel receives many booking records over time.
Users to Bookings	1:N	(Implicit) One user can make multiple bookings.
Rooms to Bookings	1:N	(Implicit) One room can be booked multiple times (on different dates).

3. Critical Observations

1. **Missing Tables:** Your SQL script references user_id and room_id in the bookings table, but the definitions for the users and rooms tables are missing.
2. **Normalization:** The amenities column in the hotels table currently stores a comma-separated string (e.g., "WiFi, Pool, Spa"). For a strictly normalized Relational Diagram (1NF), this should ideally be moved to a separate amenities table and a hotel_amenities junction table.

Show your database diagram

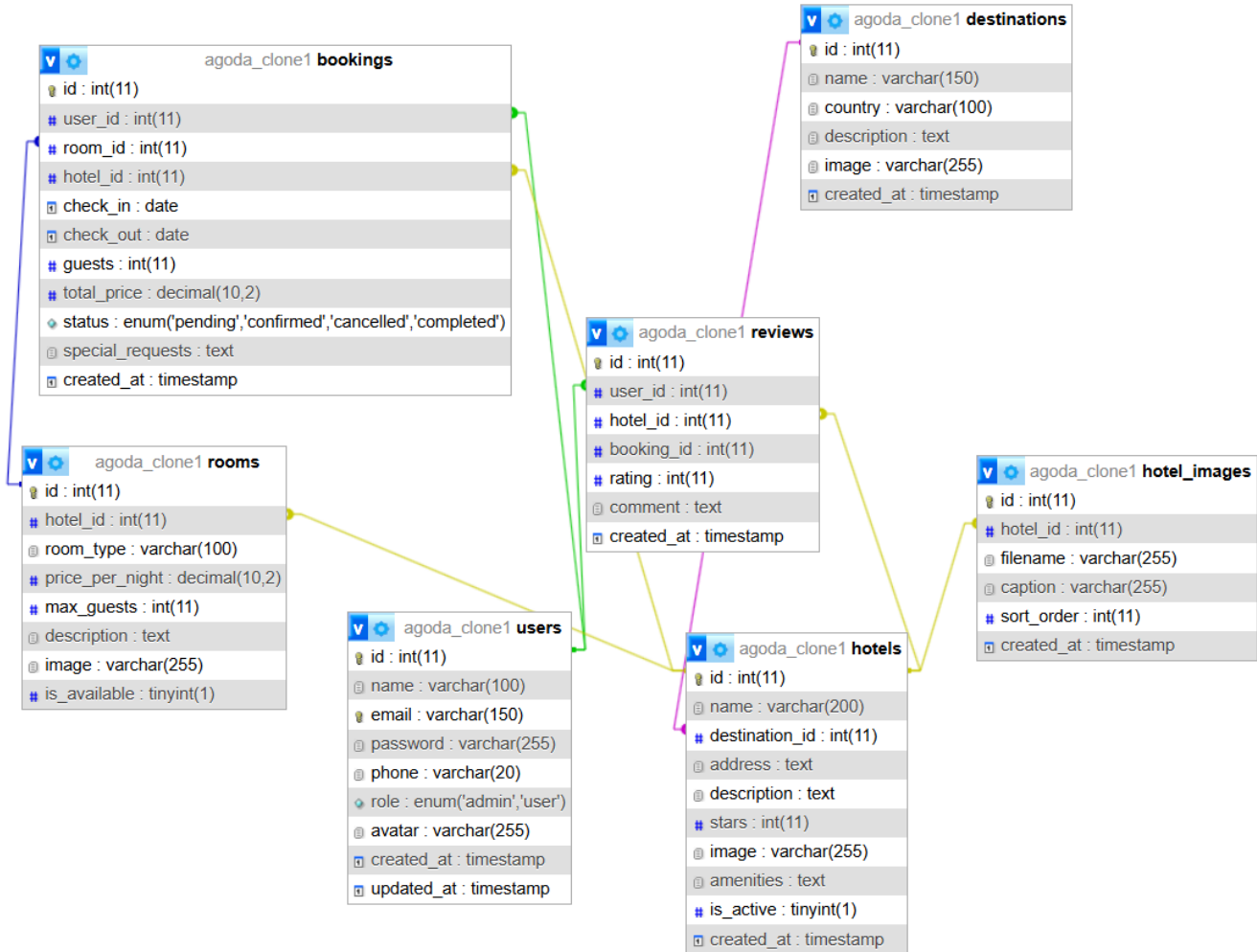


Figure 14: Database diagram

The specific functionality that the system have includes:

a. Login and Register

a. Login and Register

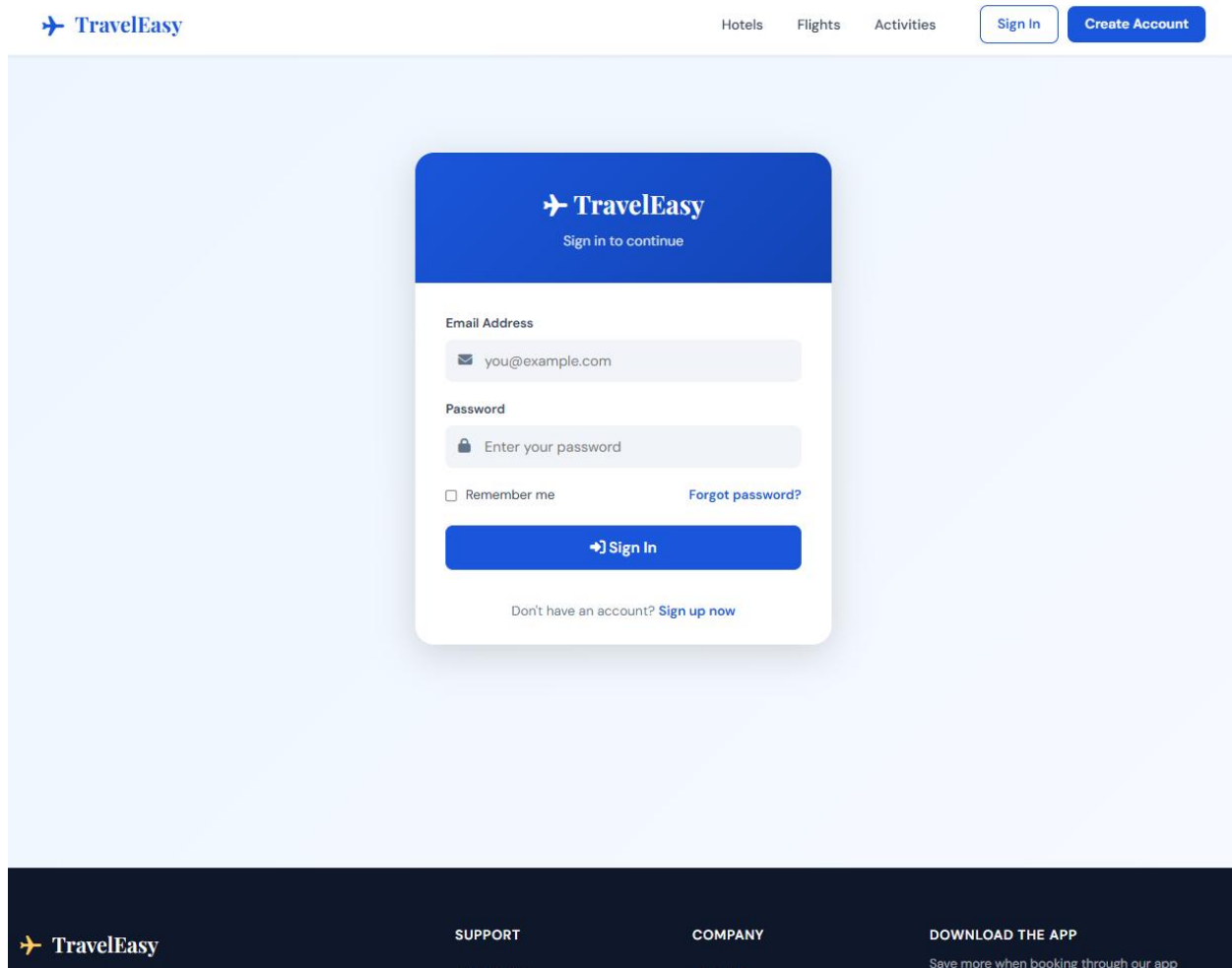
The system provides authentication functionality that allows users to create accounts and securely access the platform.

• Register:

- Users can create a new account by entering personal information such as email, username, and password.
- The system validates input data before storing it.

- After successful registration, users can log in to access services.
- **Login:**
 - Registered users can log into the system using their credentials.
 - The system verifies user information and grants access to personalized features.
 - Supports basic security such as password protection.

This functionality ensures secure access and enables personalized user experiences.



The image shows a screenshot of the TravelEasy landing page. At the top, there is a navigation bar with the TravelEasy logo on the left and links for Hotels, Flights, and Activities in the center. On the right side of the navigation bar are two buttons: 'Sign In' and 'Create Account'. The main content area features a large, centered login form. The form has a blue header with the TravelEasy logo and the text 'Sign in to continue'. Below this, there are two input fields: 'Email Address' with the placeholder 'you@example.com' and 'Password' with the placeholder 'Enter your password'. There is a checkbox for 'Remember me' and a link for 'Forgot password?'. A large blue 'Sign In' button is positioned below the password field. At the bottom of the form, there is a link that says 'Don't have an account? Sign up now'. The footer of the page is dark blue and contains the TravelEasy logo on the left, and links for 'SUPPORT', 'COMPANY', and 'DOWNLOAD THE APP' on the right. Below the 'DOWNLOAD THE APP' link, there is a small text that says 'Save more when booking through our app'.

Figure 15: Landing page login

The image shows a web browser displaying the TravelEasy landing page. At the top, there are logos for BTEC (Alliance with Education) and Pearson. Below these, the TravelEasy logo is on the left, and navigation links for Hotels, Flights, and Activities are in the center. On the right, there are 'Sign In' and 'Create Account' buttons. The main content area features a large, centered registration form. The form has a blue header with the TravelEasy logo and the text 'Create a new account'. Below this, there are input fields for 'Full Name' (with a red asterisk), 'Email Address' (with a red asterisk), 'Phone Number', 'Password' (with a red asterisk and a lock icon indicating a minimum of 6 characters), and 'Confirm Password' (with a red asterisk and a lock icon indicating to re-enter the password). A link for 'Terms of Use and Privacy Policy' is provided. At the bottom of the form is a blue 'Create Account' button with a user icon. Below the button, it says 'Already have an account? Sign in'.

Figure 16: Landing page register

b. Landing Page

The landing page is the main interface that users see when accessing the website.

- Displays key services such as:
 - Flight booking(coming soon)
 - Hotel booking
- Provides search functionality:
 - Users can search for flights by departure, destination, date, and number of passengers
- Shows promotions, featured destinations, and travel information.
- Includes navigation menu for easy access to different sections (Login, Register, Services).

The landing page plays an important role in attracting users and providing quick access to core features.

HotelsFlightsActivitiesSign InCreate Account

OVER 2 MILLION HOTELS WORLDWIDE

Travel the World, Without Breaking the Bank

Find and book the best hotels at the lowest prices, guaranteed.

HotelsFlightsHomes & ApartmentsFlight + HotelActivities

DESTINATION

CHECK-IN

CHECK-OUT

Search

2M+ Hotels

200+ Countries

50M+ Customers

4.8/5 Rating

24/7 Support

Figure 17: Landing page dashboard

Popular Destinations

[View all →](#)



Most Loved Hotels

[View all →](#)

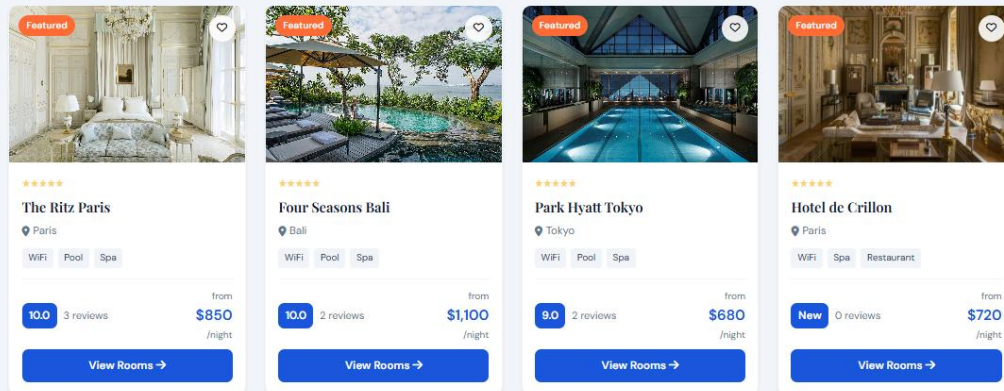


Figure 18: Landing page dashboard

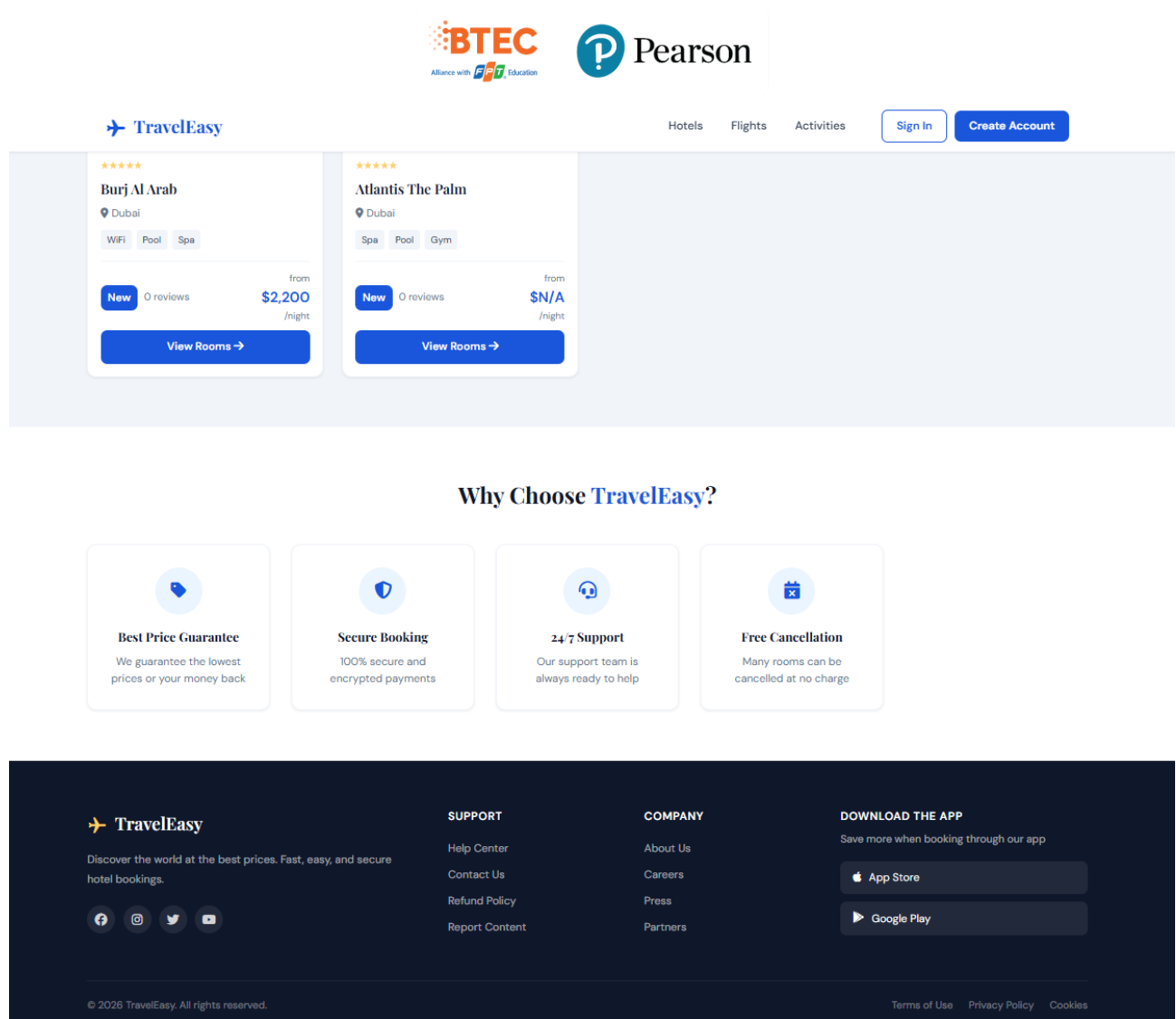


Figure 19: Landing page dashboard

c. User Management

The system allows management of user information and accounts.

- **Profile Management:**
 - Users can view and update personal information (name, email, password).
- **Account Management (Admin side):**
 - Admin can manage user accounts (view, update, delete if needed).
 - Control user roles (e.g., admin, customer).
- **User Data Storage:**

- All user data is stored securely in the database.

This functionality helps maintain user data, supports system administration, and improves user experience.

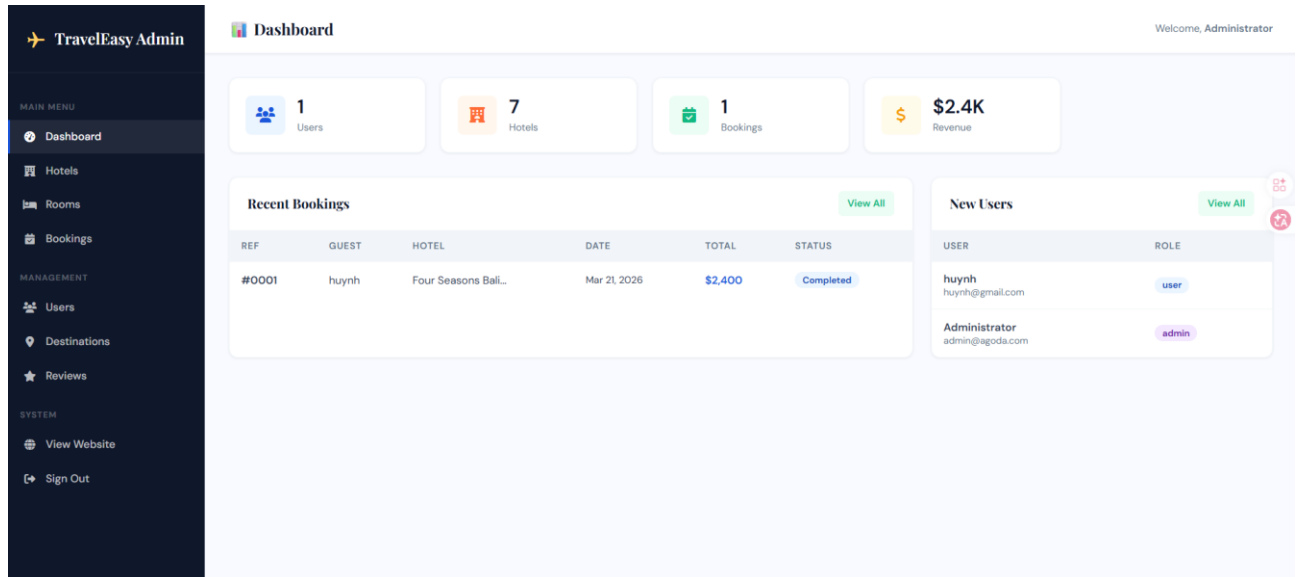


Figure 20: Landing page admin dashboard

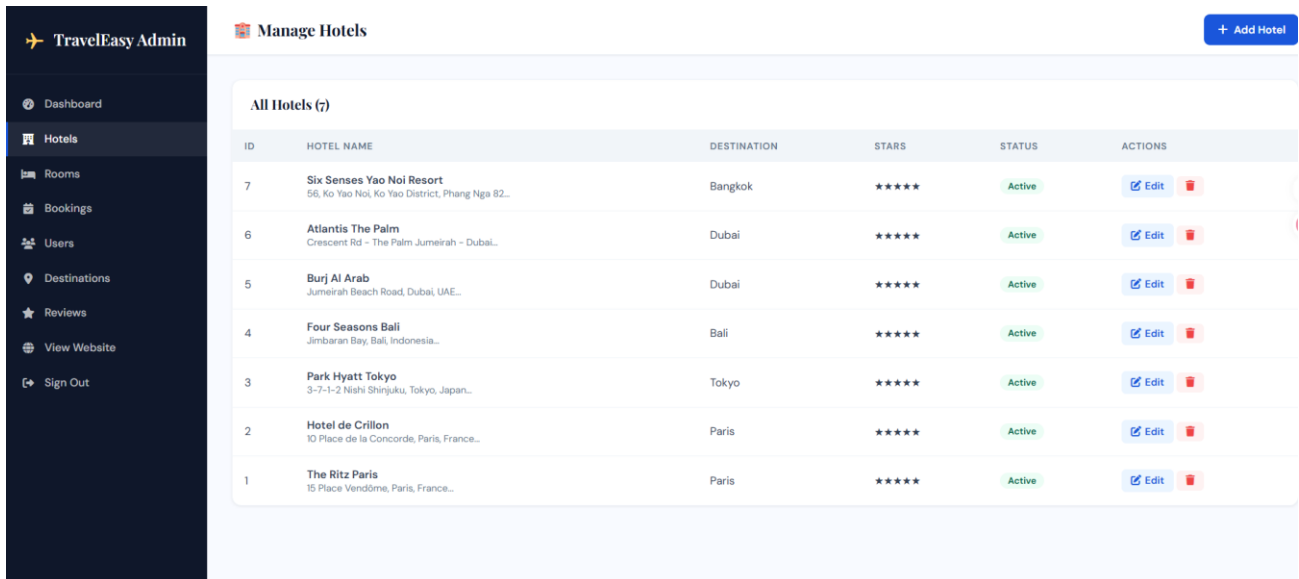


Figure 21: Landing page manage hotels

TravelEasy Admin

- Dashboard
- Hotels
- Rooms**
- Bookings
- Users
- Destinations
- Reviews
- View Website
- Sign Out

Manage Rooms

+ Add Room

All Rooms (11)

All Hotels

ID	HOTEL	ROOM TYPE	PRICE/NIGHT	MAX GUESTS	STATUS	ACTIONS
3	The Ritz Paris...	Imperial Suite Lavish suite with a living room, dining area, and butle...	\$4,500	4 guests	Available	Edit Delete
2	The Ritz Paris...	Deluxe Room Spacious room with a private balcony overlooking Place...	\$1,200	2 guests	Available	Edit Delete
1	The Ritz Paris...	Classic Room Elegant room with classic Parisian décor and garden vi...	\$850	2 guests	Available	Edit Delete
5	Hotel de Crillon...	Junior Suite Suite with a separate seating area and marble bathroom...	\$1,450	2 guests	Available	Edit Delete
4	Hotel de Crillon...	Superior Room Refined room with period furnishings and high ceilings...	\$720	2 guests	Available	Edit Delete
7	Park Hyatt Tokyo...	Park Suite Expansive suite with floor-to-ceiling windows and priva...	\$1,800	3 guests	Available	Edit Delete
6	Park Hyatt Tokyo...	Park Room Contemporary room with stunning Shinjuku skyline views...	\$680	2 guests	Available	Edit Delete
9	Four Seasons Bali...	Ocean View Villa Stunning villa with panoramic Jimbaran Bay ocean views...	\$2,400	4 guests	Available	Edit Delete
8	Four Seasons Bali...	Garden Villa Private villa surrounded by tropical gardens with plung...	\$1,100	2 guests	Available	Edit Delete
11	Burj Al Arab...	Royal Suite The ultimate in luxury -- an entire floor with 360° v...	\$8,500	6 guests	Available	Edit Delete

Figure 22: Landing page rooms

TravelEasy Admin

- Dashboard
- Hotels
- Rooms
- Bookings**
- Users
- Destinations
- Reviews
- View Website
- Sign Out

Manage Bookings

0 Pending

0 Confirmed

0 Cancelled

1 Completed

All Bookings (1)

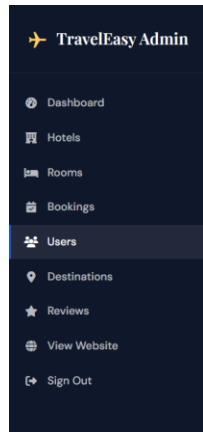
Search guest or hotel...

All Statuses

Filter

REF	GUEST	HOTEL / ROOM	CHECK-IN	CHECK-OUT	TOTAL	STATUS	ACTIONS
#00001	huynh huyinh@gmail.com	Four Seasons Bali... Ocean View Villa - 1 guest	Mar 21, 2026	Mar 22, 2026	\$2,400	Completed	Delete

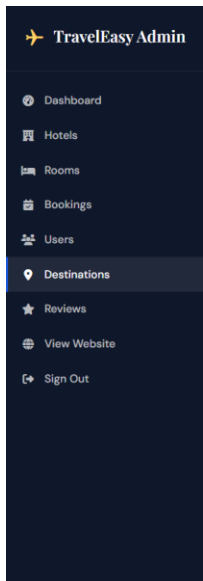
Figure 23: Landing page admin bookings



Manage Users

All Users (2)						
Search by name or email...						
ID	USER	PHONE	BOOKINGS	JOINED	ROLE	ACTIONS
2	huynh huynh@gmail.com	0338251963	1	Mar 19, 2026	User	Edit Delete
1	Administrator admin@agoda.com	—	0	Mar 16, 2026	Admin	You

Figure 24: Landing page admin users

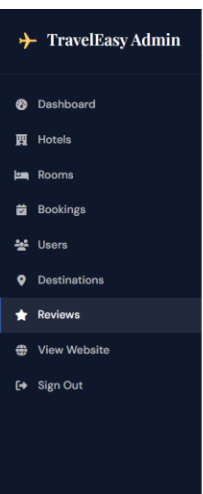


Manage Destinations

+ Add Destination

All Destinations (8)						
ID	DESTINATION	COUNTRY	DESCRIPTION	HOTELS	ACTIONS	
11	London	United Kingdom	Historic capital with iconic landmarks and rich culture...	0 hotels	Edit	Delete
9	Ha Noi	Viet Nam	Hanoi, the capital of Vietnam, is renowned for its centuries-old...	0 hotels	Edit	Delete
7	Bangkok	Thai Lan	Thailand, or simply Thai, officially the Kingdom of Thailand, is...	1 hotel	Edit	Delete
5	Dubai	UAE	A gleaming desert metropolis with record-breaking architecture an...	2 hotels	Edit	Delete
4	Bali	Indonesia	Tropical paradise known for lush rice terraces, vibrant nightlife...	1 hotel	Edit	Delete
3	New York	USA	The city that never sleeps — iconic skyline, Broadway shows, an...	0 hotels	Edit	Delete
2	Tokyo	Japan	A dazzling blend of futuristic technology and ancient temples in...	1 hotel	Edit	Delete
1	Paris	France	The City of Light, famed for the Eiffel Tower, world-class cuisin...	2 hotels	Edit	Delete

Figure 25: Landing page admin destinations



Manage Reviews

All Reviews (3)						
ID	GUEST	HOTEL	SCORE	COMMENT	DATE	ACTIONS
1	Administrator	The Ritz Paris...	10/10	Absolutely flawless. From the moment we arrived, every detail was perfect. The s...	Mar 16, 2026	Delete
2	Administrator	Park Hyatt Tokyo...	9/10	The views from the Park Hyatt are simply breathtaking. Rooms are immaculate and...	Mar 16, 2026	Delete
3	Administrator	Four Seasons Bali...	10/10	Waking up in a private villa in Bali with a plunge pool — there are no words...	Mar 16, 2026	Delete

Figure 26: Landing page admin review

M2. Discuss the components of a feasibility report.

A feasibility report evaluates whether a proposed system or project is practical, achievable, and beneficial for an organization. It helps decision-makers determine whether to proceed, modify, or cancel the project before investing significant resources.

The three core components of a feasibility report are:

1. Economic Feasibility
2. Technical Feasibility
3. Organizational Feasibility

1. Economic Feasibility

Economic feasibility evaluates whether a project is financially viable and worth the investment. It focuses on comparing the total cost of developing and operating the system with the expected financial and business benefits. The goal is to determine whether the organization will gain value from implementing the system.

One of the most important elements of economic feasibility is **cost estimation**. This includes all expenses required to build, implement, and maintain the system. Development costs may include software tools, hardware infrastructure, employee salaries, training, testing, and deployment. Operational costs such as maintenance, upgrades, technical support, and hosting must also be considered. Hidden costs, such as downtime during implementation or productivity loss during training, should also be included to ensure realistic financial planning.

Another critical aspect is **benefit analysis**. Benefits can be tangible or intangible. Tangible benefits are measurable in monetary terms, such as increased revenue, reduced operational costs, improved efficiency, and decreased manual workload. Intangible benefits include improved customer satisfaction, better decision-making through data analytics, enhanced brand reputation, and competitive advantage. Although intangible benefits are difficult to measure financially, they still contribute significantly to long-term organizational success.

Economic feasibility also examines financial evaluation methods such as **payback period**, **return on investment (ROI)**, and **net present value (NPV)**. The payback period measures how long it takes to recover the initial investment. ROI evaluates profitability relative to the cost of investment. NPV calculates the value of future cash flows compared to present costs, helping organizations assess long-term financial value.

Risk assessment is also important in economic feasibility. Unexpected expenses, market changes, or lower-than-expected system usage may affect financial returns. Therefore, contingency planning should be included.

In summary, economic feasibility ensures that the project is financially justified. If projected benefits outweigh costs within an acceptable timeframe, the project can be considered economically viable. Otherwise, alternative solutions or adjustments may be required before proceeding.

2. Technical Feasibility

Technical feasibility evaluates whether the organization has the technical capability, resources, and infrastructure required to successfully develop and operate the proposed system. It determines whether the project is technologically achievable and sustainable in the long term.

One of the first factors considered is **technology availability**. This includes hardware, software platforms, development tools, network infrastructure, and system architecture. The organization must determine whether existing systems can support the new solution or if upgrades are required. Compatibility with current databases, operating systems, and third-party services must also be evaluated to avoid integration problems.

Another key factor is **technical expertise**. The development team must possess the necessary knowledge and skills to design, build, test, and maintain the system. If the required expertise is not available internally, the organization may need to hire specialists, train employees, or outsource development. These decisions affect both cost and project timeline.

Technical feasibility also includes **system performance and scalability**. The system must be capable of handling expected workloads, user traffic, and data volume. It should perform reliably under peak demand and allow future expansion as the organization grows. Poor scalability may result in performance failures or costly redesign later.

Security and reliability are also essential considerations. The system must protect sensitive data, prevent unauthorized access, and ensure stable operation. Backup mechanisms, disaster recovery plans, and fault tolerance must be evaluated to minimize system failure risks.

Technical risk assessment is another important aspect. Potential risks include technology limitations, integration challenges, insufficient testing tools, or rapid technological changes that may make the system obsolete.

In conclusion, technical feasibility determines whether the project can be successfully implemented using available technology and skills. If the organization has the required infrastructure, expertise, and

resources, the project is considered technically feasible. Otherwise, technical limitations may require redesign or alternative solutions.

3. Organizational Feasibility

Organizational feasibility evaluates how well the proposed system fits within the organization's structure, culture, and operational environment. It determines whether the organization is prepared to support, adopt, and effectively use the new system.

A major aspect of organizational feasibility is **management support**. Successful system implementation requires strong leadership commitment. Management must provide resources, approve policies, and guide organizational change. Without executive support, projects often face resistance or lack proper direction.

Another critical factor is **user acceptance**. Employees and stakeholders must be willing to adopt the new system and change existing work processes. Resistance to change is a common challenge, especially if the system alters daily tasks or requires new skills. Proper communication, training programs, and involvement of users in the development process can increase acceptance.

Organizational feasibility also examines **operational impact**. The system must align with current business processes or improve them effectively. If major workflow changes are required, the organization must be capable of restructuring procedures and responsibilities. The system should enhance productivity rather than create confusion or inefficiency.

Human resource readiness is also important. Staff must have the skills to operate and maintain the system. Training requirements, workload adjustments, and role changes must be considered to ensure smooth transition.

Organizational culture also plays a role. Some organizations adapt quickly to technological innovation, while others prefer stability and gradual change. The system must fit the organization's working style and strategic goals.

Finally, organizational feasibility considers long-term sustainability. The organization must be able to maintain, support, and continuously improve the system after implementation.

In summary, organizational feasibility determines whether the system is acceptable and practical within the organization. If the system aligns with business processes, has management support, and is accepted by users, it is considered organizationally feasible.

M3. Analyse how software requirements can be traced throughout the software lifecycle

Requirements traceability is the process of tracking and linking software requirements throughout the entire Software Development Life Cycle (SDLC). It ensures that every requirement is implemented, tested, and maintained properly.

1. Requirement Traceability in SDLC Phases

1.1 Requirements Gathering Phase

In this phase, both functional and non-functional requirements are identified and documented.

Examples:

- Users must be able to register and log in
- Users can search destinations and hotels
- Users can book rooms

These requirements are typically documented in the **Software Requirement Specification (SRS)**.

1.2 System Design Phase

At this stage, requirements are transformed into system design models, including:

- Use Case Diagrams
- Entity Relationship Diagram (ERD)
- System Architecture

Example:

Requirement: *User can book a room*

Design:

- Database table: bookings
- Relationships: users → bookings → rooms → hotels

This ensures that requirements are clearly mapped into the system structure.

1.3 Implementation Phase (Development)

During implementation, each requirement is translated into specific code modules.

Examples:

Requirement	Implementation
Login/Register	Authentication module (login.php)
Booking	booking.php, bookings table
User Management	admin_user.php

Traceability ensures that:

- All requirements are implemented
- No unnecessary features are developed

1.4 Testing Phase

Each requirement must be verified through corresponding test cases.

Examples:

Requirement	Test Case
User login	TC01 – Valid login
Booking	TC05 – Successful booking
Review	TC08 – Submit review

If a test fails, developers can trace back to the related requirement and implementation to fix the issue.

1.5 Deployment Phase

In this phase, the system is deployed to a live environment.

For the TravelEasy system:

- The website is deployed online
- Features such as login, booking, and search are tested in real conditions

Traceability ensures that deployed features meet the original requirements.

1.6 Maintenance Phase

After deployment, the system may require updates or improvements.

Example:

- Adding a “Cancel Booking” feature

Traceability helps identify:

- Which modules need modification
- Which database tables are affected (e.g., bookings)
- Which test cases need to be updated

2. Requirement Traceability Matrix (RTM)

A Requirement Traceability Matrix (RTM) is used to link requirements with design, implementation, and testing.

Example:

Requirement	Design	Implementation	Test Case
Login	Use Case Diagram	login.php	TC01
Booking	ERD (bookings)	booking.php	TC05
Review	reviews table	review.php	TC08

The RTM ensures that every requirement is tracked throughout the system.

3. Benefits of Requirement Traceability

- Ensures all requirements are fully implemented
- Improves software quality and reliability
- Helps identify missing or redundant features
- Supports system maintenance and upgrades
- Reduces development risks

M4. Discuss two approaches to improving software quality

1. Introduction

Software quality is an important factor in ensuring that a system works correctly, efficiently, and meets user requirements. In the TravelEasy project, two main approaches are used to improve software quality:

- Improving **software functionality** using a **Test Plan**
- Improving **software structure** using **Flowcharts**

◆ 2. Software Functionality Quality – Using Test Plan

2.1 Purpose of Test Plan

The test plan is designed to ensure that all system features such as login, registration, and booking work correctly. It helps identify errors, validate system behavior, and improve reliability.

2.2 Test Scope

The following features are tested:

- Login and Register
- Landing Page
- Hotel Search
- Booking System
- User Management

2.3 Test Environment

- OS: Windows 10
- Browser: Google Chrome
- Backend: PHP
- Database: MySQL
- Hosting: free.nf

2.4 Test Cases

◆ Login Function

ID	Description	Input	Expected Result
TC01	Valid login	Correct email & password	Login successful
TC02	Invalid password	Wrong password	Show error
TC03	Empty input	No data	Validation error

Register Function

ID	Description	Input	Expected Result
TC04	Valid register	Correct info	Account created
TC05	Duplicate email	Existing email	Error
TC06	Invalid email	Wrong format	Validation error

Booking Function

ID	Description	Input	Expected Result
TC07	Book hotel	Valid data	Booking success
TC08	Missing data	Incomplete form	Error
TC09	Not logged in	Guest user	Redirect to login

2.5 Test Evidence

Figure 1: Login Page Interface

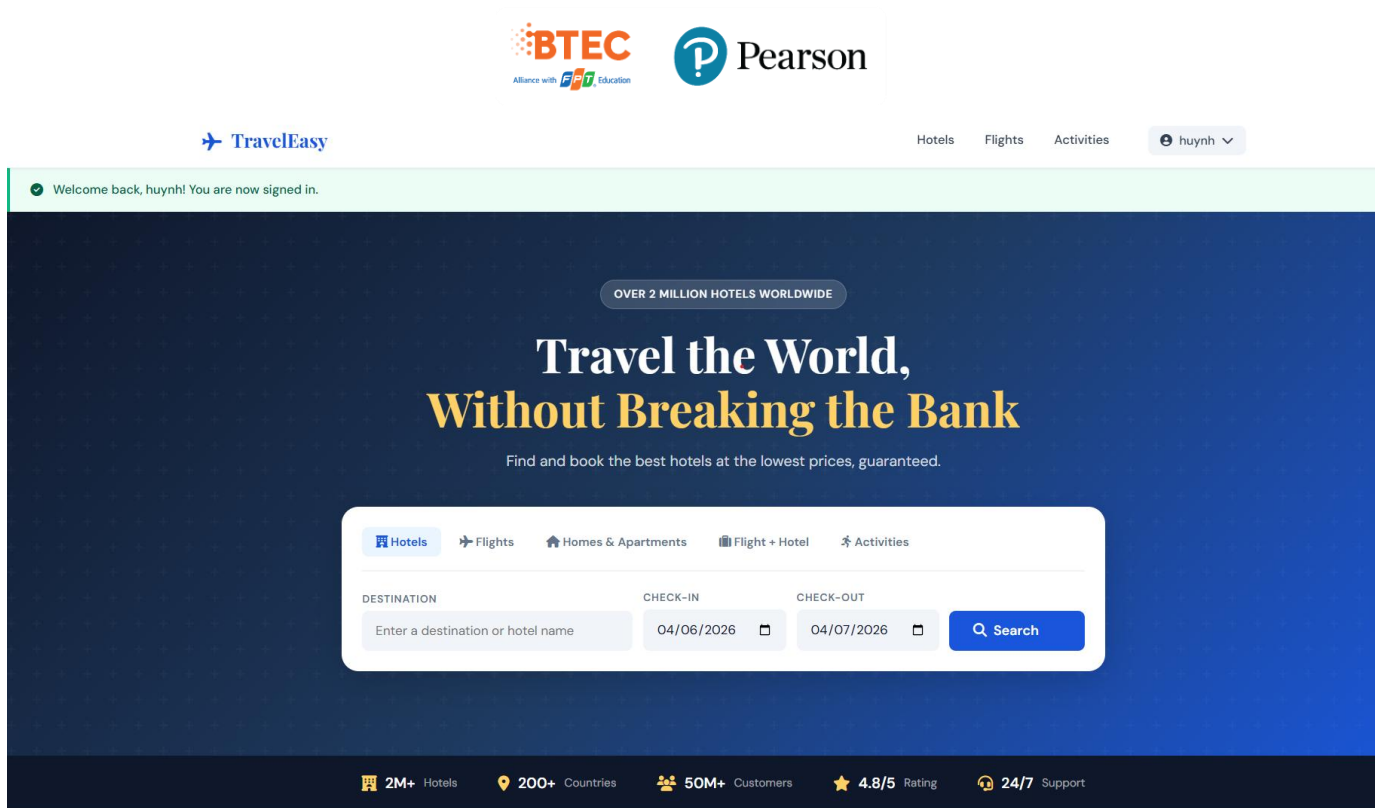


Figure 27: Test plan login

Explanation:

This figure shows the login interface where users enter their credentials to access the system.

Figure 2: Login Test – Invalid Password

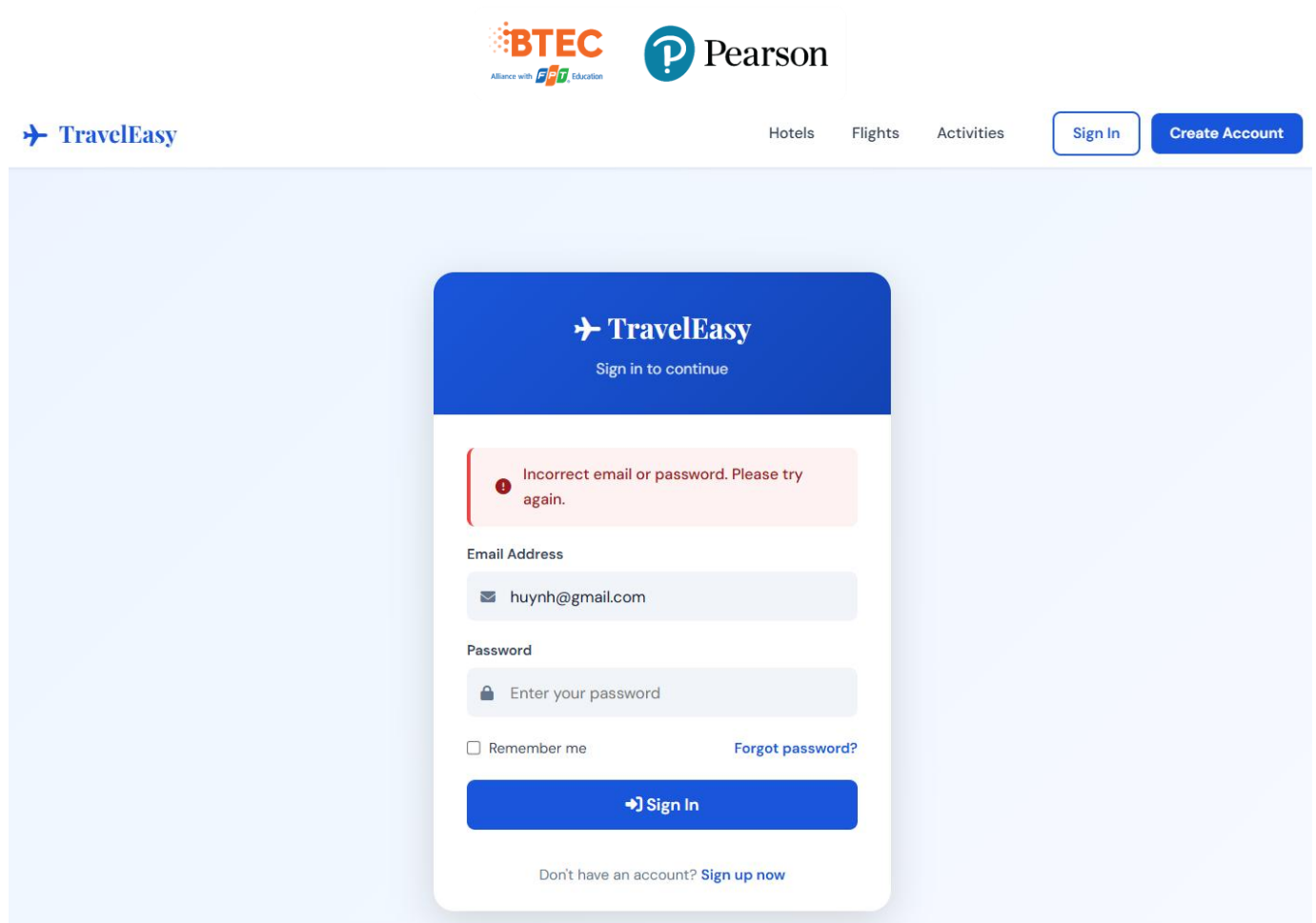


Figure 28: Test plan error

Explanation:

This test verifies that the system displays an error message when an incorrect password is entered.

Figure 3: Successful Booking

Confirm Your Booking

Booking Summary



The Ritz Paris

15 Place Vendôme, Paris, France

Room Type	Deluxe Room
Check-in	Apr 06, 2026
Check-out	Apr 07, 2026
Duration	1 night
Guests	1 guest
Rate/Night	\$1,200
Total	\$1,200

Guest Details

huynh

huynh@gmail.com

Special Requests

Non-smoking room, high floor,
extra pillows...

Free cancellation available up to 24 hours before check-in.

Confirm Booking

[← Go Back](#)

Figure 29: Test plan bookings

Booking confirmed! Your booking reference: #000002

My Bookings

Booking Ref: #000002

Pending



The Ritz Paris

Deluxe Room · 15 Place Vendôme, Paris, France

\$1,200

Check-in: Apr 06, 2026 Check-out: Apr 07, 2026 1 guest Booked: Apr 05, 2026 18:08

[View Hotel](#)

[Cancel Booking](#)

Booking Ref: #000001

Completed



Four Seasons Bali

Ocean View Villa · Jimbaran Bay, Bali, Indonesia

\$2,400

Check-in: Mar 21, 2026 Check-out: Mar 22, 2026 1 guest Booked: Mar 19, 2026 19:01

[View Hotel](#)

Figure 30: Test plan confirm booking

Explanation:

This figure demonstrates that the system successfully processes a hotel booking when valid data is provided.

Figure 4: Admin Dashboard

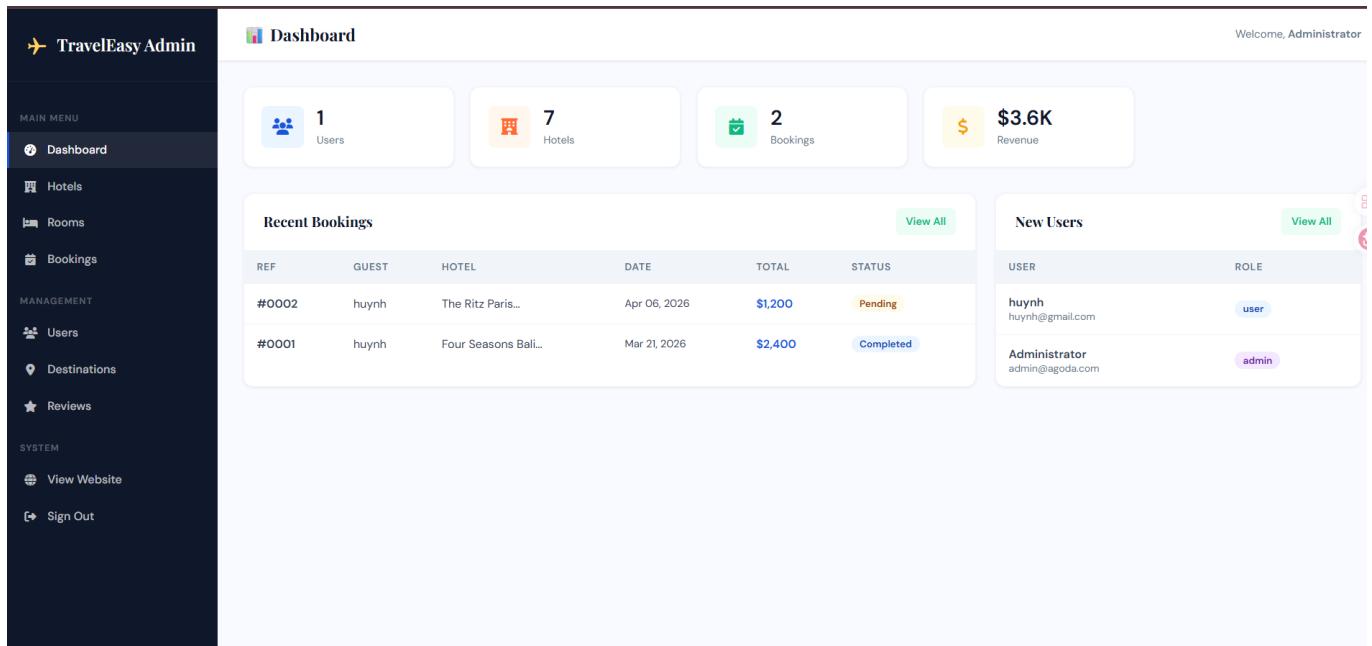


Figure 31: Test plan admin dashboard

Explanation:

This figure shows the admin dashboard interface where the administrator can manage users, bookings, and system data.

Figure 5: Admin – Manage Users

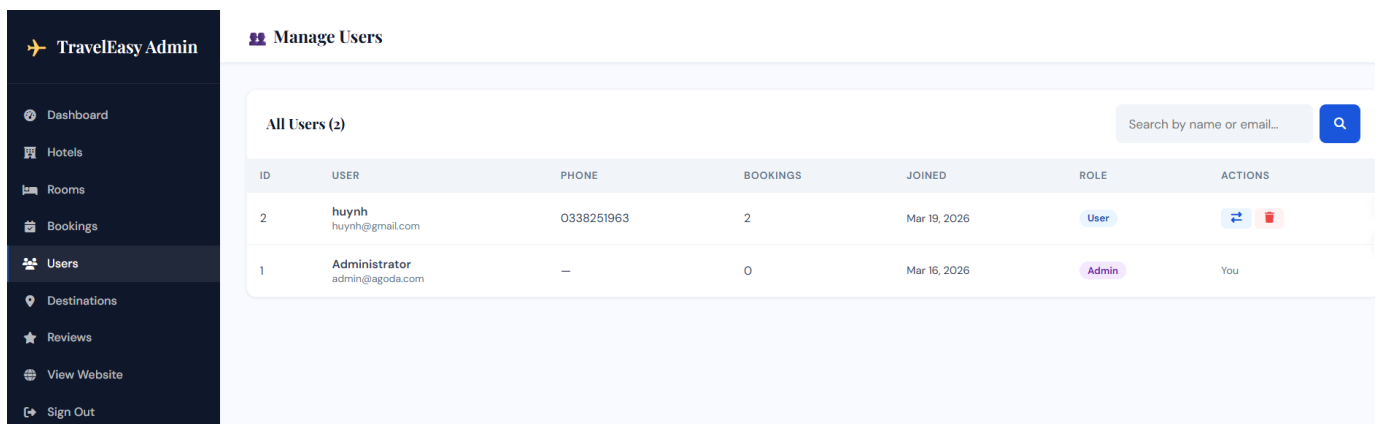
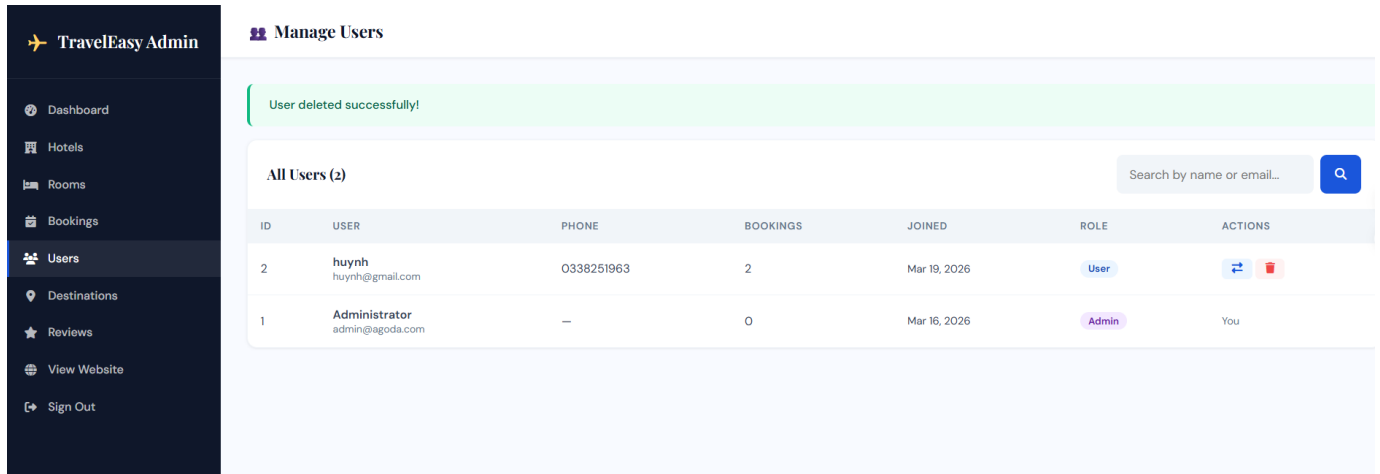


Figure 32: Test plan admin manage users

Explanation:

This test verifies that the admin can view the list of users and manage user accounts effectively.

Figure 6: Admin – Delete User



Manage Users

User deleted successfully!

All Users (2) Search by name or email...

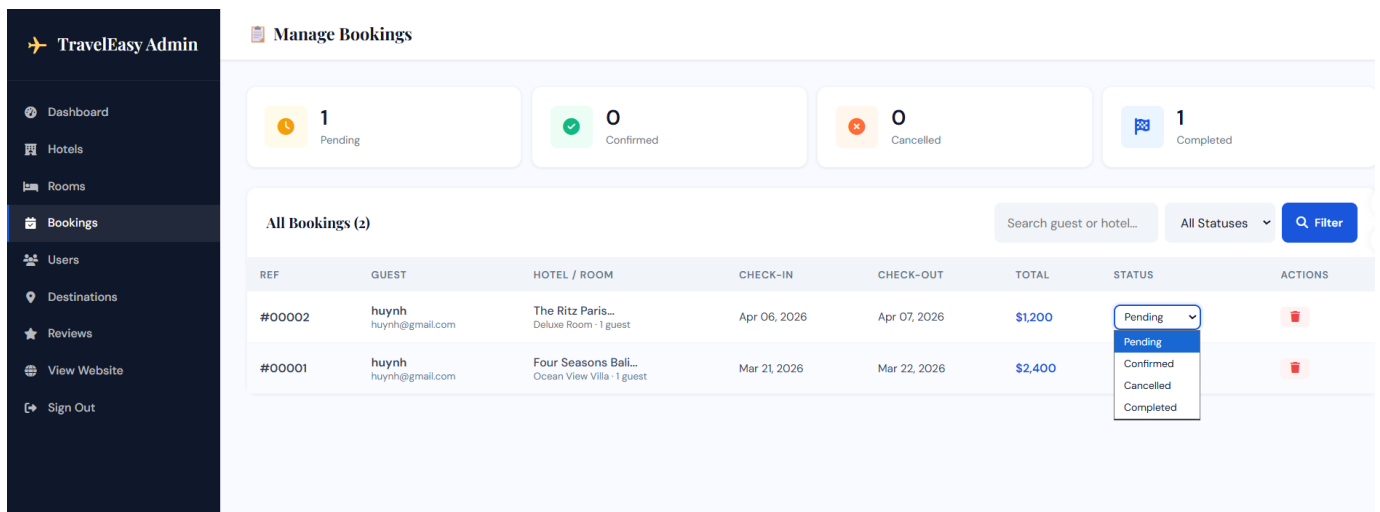
ID	USER	PHONE	BOOKINGS	JOINED	ROLE	ACTIONS
2	huynh huynh@gmail.com	0338251963	2	Mar 19, 2026	User	 
1	Administrator admin@agoda.com	—	0	Mar 16, 2026	Admin	You

Figure 33: Test plan admin delete users

Explanation:

This test ensures that the admin can delete a user account and that the system updates the database correctly.

Figure 7: Admin – Manage Bookings



Manage Bookings

1 Pending 0 Confirmed 0 Cancelled 1 Completed

All Bookings (2) Search guest or hotel... All Statuses Filter

REF	GUEST	HOTEL / ROOM	CHECK-IN	CHECK-OUT	TOTAL	STATUS	ACTIONS
#00002	huynh huynh@gmail.com	The Ritz Paris... Deluxe Room - 1 guest	Apr 06, 2026	Apr 07, 2026	\$1,200	Pending	
#00001	huynh huynh@gmail.com	Four Seasons Bali... Ocean View Villa - 1 guest	Mar 21, 2026	Mar 22, 2026	\$2,400	Confirmed	

Figure 34: Test plan admin manage bookings

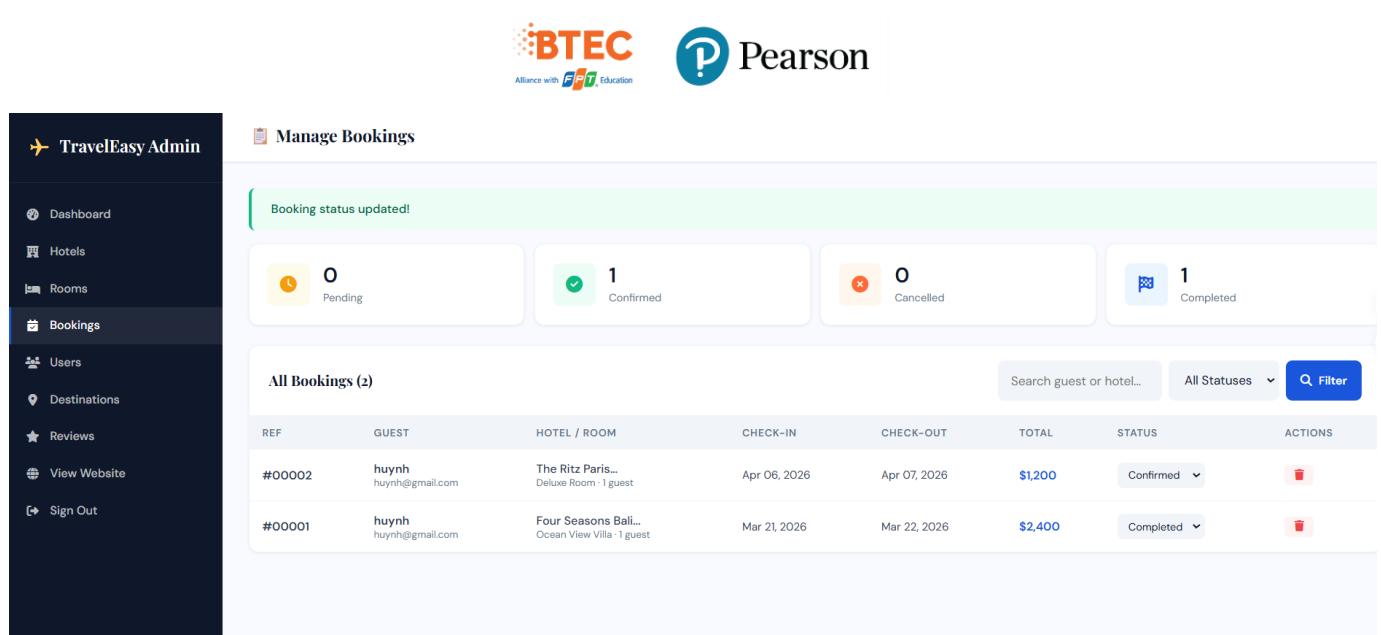


Figure 35: Test plan admin manage bookings

Explanation:

This figure demonstrates that the admin can view and manage all booking records in the system.

3. Software Structure Quality – Using Flowcharts

Login Flowchart

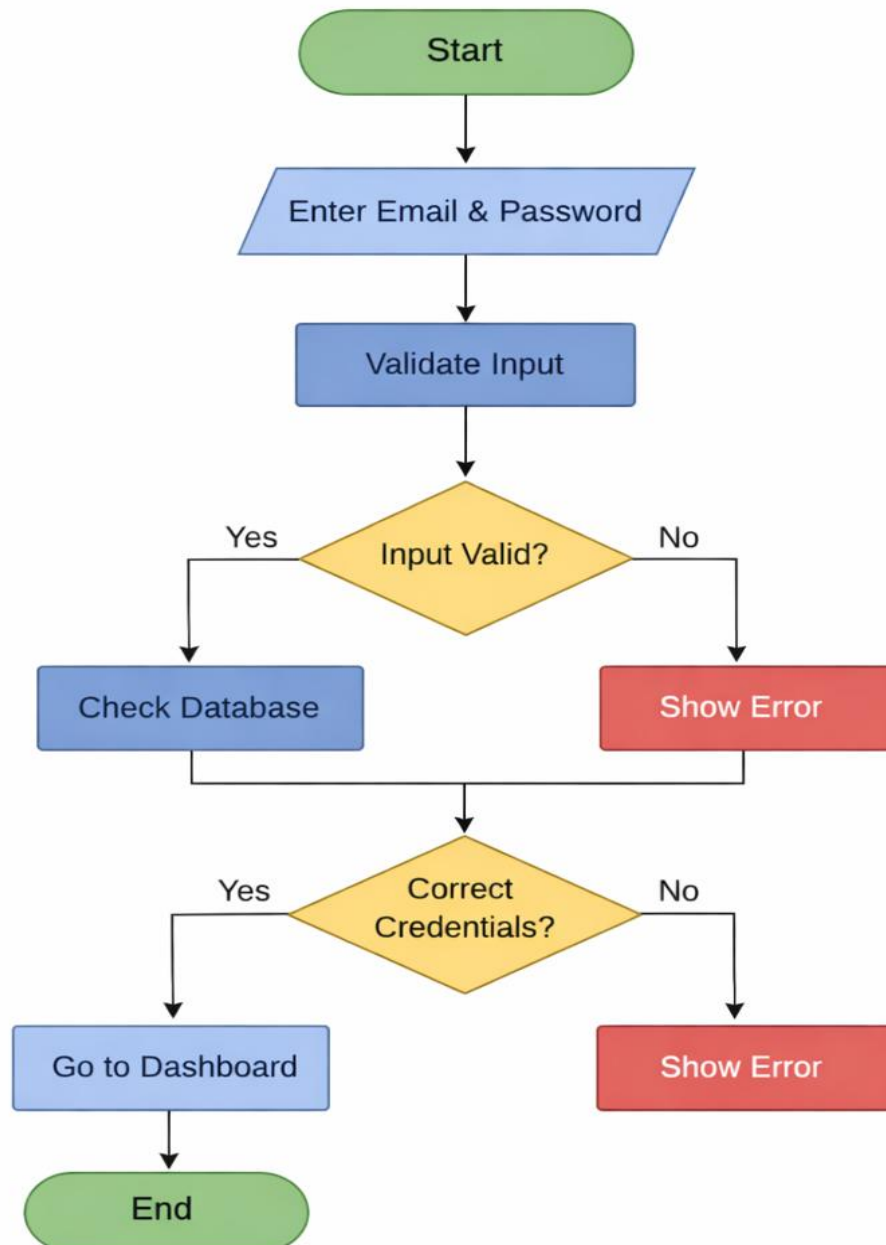


Figure 36: Login flowchart

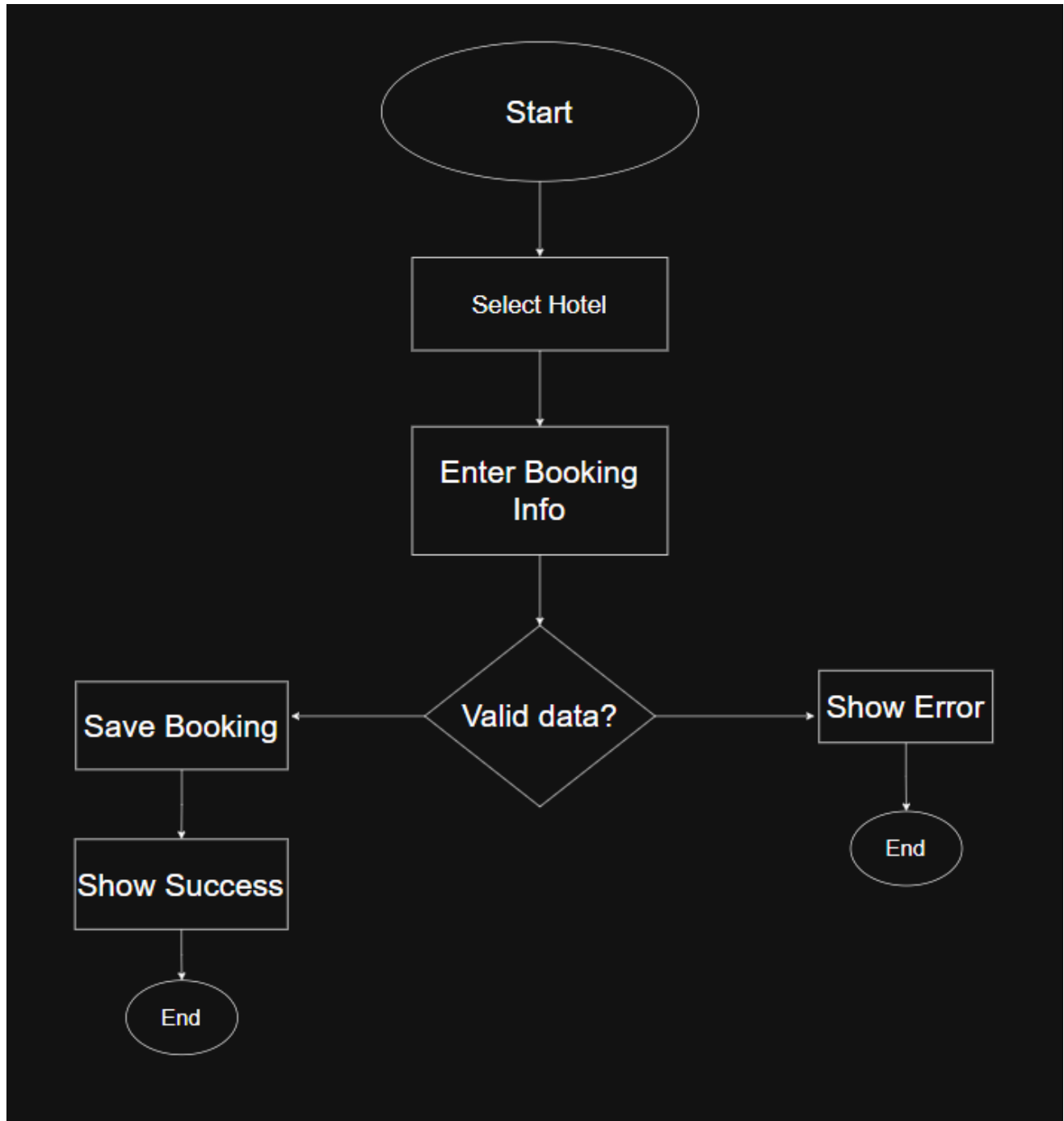


Figure 37: Booking flowchart

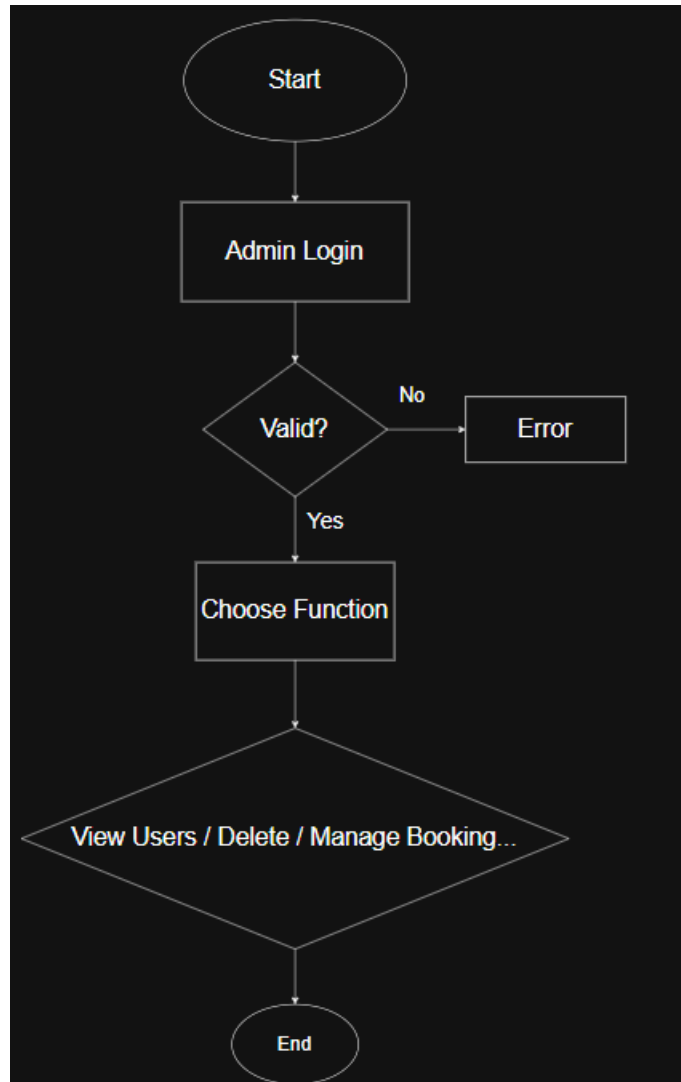


Figure 38: Admin flowchart

M5. Analyse a range of software behavioural tools and techniques

1. Use Case Diagram

Advantages:

- Helps identify system functionalities from the user's perspective
- Easy to understand for both developers and non-technical stakeholders
- Clearly shows interaction between actors (User, Admin) and the system
- Supports requirement analysis effectively

Disadvantages:

- Does not describe internal processing logic
- Cannot represent data flow or system structure
- Becomes complex when there are too many use cases

Limitations in this project:

- Only useful in the analysis phase, not directly applied in coding
- Cannot represent backend logic such as PHP processing or database queries
- Does not show detailed workflows like login or booking implementation

2. Data Flow Diagram (DFD)**Advantages:**

- Clearly shows how data moves through the system
- Helps understand interaction between user, system, and database
- Useful for designing backend logic and database structure

Disadvantages:

- Does not represent control flow such as conditions or loops
- Can become confusing when expanded into multiple levels
- Not suitable for describing user interface behavior

Limitations in this project:

- Cannot represent HTML interfaces or UI interactions
- Does not show detailed validation logic in PHP
- Mainly used for system design, not implementation

3. Finite State Machine (FSM)**Advantages:**

- Clearly represents system states and transitions
- Useful for modeling login status and booking status

- Helps ensure logical consistency in system behavior

Disadvantages:

- Becomes complex with many states and transitions
- Difficult for beginners to design and understand
- Not suitable for very large systems

Limitations in this project:

- PHP and HTML follow request-response model, not state-driven
- Difficult to implement fully without JavaScript or advanced session handling
- Mainly used for conceptual modeling rather than real implementation

4. Technical Limitations of the Project**Using HTML:**

- Only supports static user interface
- Cannot process logic or validation
- Cannot interact with database

Using Pure PHP:

- Works on server-side only
- Stateless (each request is independent)
- Difficult to handle complex state transitions

Without Framework:

- No clear structure like MVC
- Code becomes repetitive and harder to maintain
- Difficult to scale the system

Limited JavaScript:

- Reduces user interaction and experience

- No real-time updates
- Cannot simulate dynamic behaviors

M6. Differentiate between a finite state machine (FSM) and an extended FSM, providing an application for both

1. Finite State Machine (FSM)

Definition:

- A Finite State Machine (FSM) is a computational model used to represent a system with a finite number of states and transitions between those states.
- The system changes from one state to another based on input events.

Characteristics:

- Consists of:
 - States
 - Transitions
 - Inputs
- No memory of past data except the current state
- Transitions depend only on input conditions

Advantages:

- Simple and easy to understand
- Suitable for modeling basic system behavior
- Easy to implement

Disadvantages:

- Cannot handle complex logic
- No support for variables or data storage
- Difficult to scale for large systems

2. Extended Finite State Machine (EFSM)

Definition:

- An Extended Finite State Machine (EFSM) is an advanced version of FSM that includes variables, conditions, and actions.
- It allows transitions to depend not only on inputs but also on data values.

Characteristics:

- Includes all FSM components plus:
 - Variables (data storage)
 - Guards (conditions for transitions)
 - Actions (operations during transitions)
- Can store and process data

Advantages:

- Can model complex systems
- Supports conditions and data processing
- More flexible and powerful than FSM

Disadvantages:

- More difficult to design and understand
- Higher complexity
- Harder to implement compared to FSM

3. Differences between FSM and EFSM

- FSM does not support variables, while EFSM supports variables and data storage
- FSM transitions depend only on inputs, while EFSM transitions depend on both inputs and conditions (guards)
- FSM is simple and suitable for small systems, while EFSM is more suitable for complex systems

- FSM cannot handle detailed logic, while EFSM can model real-world scenarios with conditions and actions

4. Real-world Examples

FSM Examples:

- Traffic light system:
 - States: Red, Yellow, Green
 - Transitions based on timer
- User login status:
 - States: Logged out → Logged in
 - Transition occurs after successful login
- ATM machine (basic):
 - States: Idle → Card Inserted → Processing → Completed

EFSM Examples:

- Online booking system (your project):
 - States: Select service → Enter information → Confirm booking
 - Variables: user data, booking date, payment info
 - Conditions: valid input, available slots
- E-commerce checkout:
 - States: Cart → Checkout → Payment → Order complete
 - Variables: total price, user address
 - Conditions: payment success or failure
- Login system with validation:
 - FSM: Logged out → Logged in
 - EFSM:

- Checks username/password
- Limits login attempts
- Stores session data

Conclusion

This report has presented a comprehensive analysis of the Travel website project across the software development lifecycle. It covered key aspects including system requirements, behavioural design techniques, software quality improvement approaches, and modelling methods such as Use Case Diagram, DFD, FSM, and EFSM.

Through the analysis, it is clear that each technique plays an important role in supporting system development. Behavioural tools like flowcharts, pseudocode, and FSM help visualize system logic and workflows, while test plans ensure that the system functions correctly and meets user requirements.

The comparison between FSM and EFSM highlights the importance of choosing appropriate modelling techniques. While FSM is suitable for simple systems, EFSM provides better support for real-world applications that require data handling and conditional logic, such as login and booking processes in this project.

Additionally, the report has identified limitations related to the use of HTML and pure PHP, which affect system scalability, structure, and interactivity. This emphasizes the need for better architectural practices and possibly modern frameworks in future development.

Overall, the application of these techniques has improved both the quality of software functionality and structure, making the system more reliable, maintainable, and easier to understand.

References

- CMCglobal. (2023). *Risks Management in Software Engineering: Best Practices for Successful Project Delivery*. <https://cmcglobal.com.vn/it-outsourcing/risk-management-software-engineering/>.
- Geeksforgeeks. (2026). *Software Development Models - SDLC Models*. <https://www.geeksforgeeks.org/software-engineering/sdlc-models-types-phases-use/>.
- Lucidchart. (2026). *UML Use Case Diagram Tutorial*. <https://www.lucidchart.com/pages/tutorial/uml-use-case-diagram>.
- Miro. (2026). *How to draw an ER Diagram: Complete step-by-step guide*. <https://miro.com/diagramming/how-to-draw-an-er-diagram/>.